



# SISTEMAS BIOINSPIRADOS

*MAGÍSTER EN INTELIGENCIA  
ARTIFICIAL  
AGOSTO 2026*

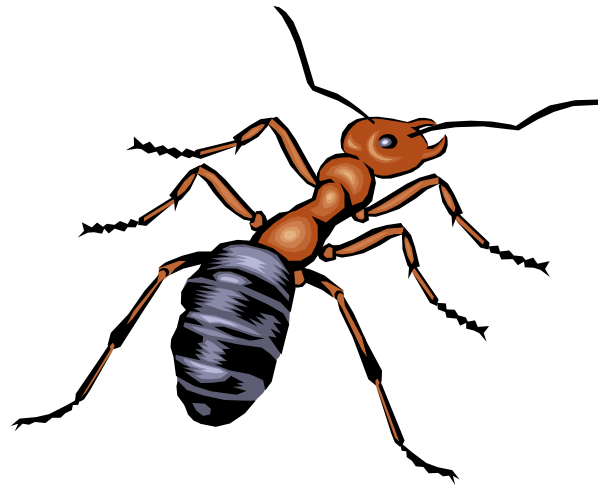
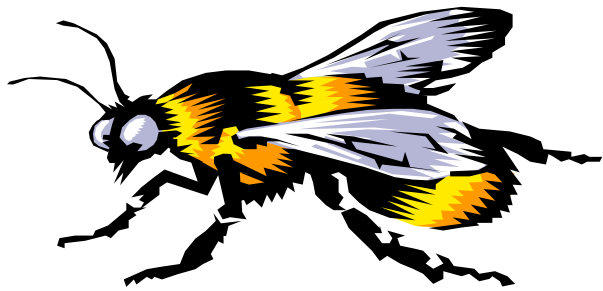
PROFESOR: RICARDO CONTRERAS A.

# THE FIRST MEN IN THE MOON

- H. G. Wells
- Publicado por George Newnes en 1901.
- Tapa de la primera edición



# SWARM INTELLIGENCE



# INTELIGENCIA DE ENJAMBRE

- Se le llama así al comportamiento de sistemas auto-organizados, descentralizados, ya sean naturales o artificiales.
- Estos sistemas consisten de una población de agentes simples, que interactúan localmente con otros agentes y con el ambiente.
- Los agentes siguen reglas muy simples, no hay una estructura de control centralizado.
- Las interacciones entre los agentes llevan a la emergencia de un comportamiento global *inteligente*, desconocido para los agentes individuales.

# ENTRE LAS VARIANTES MÁS POPULARES ESTÁN

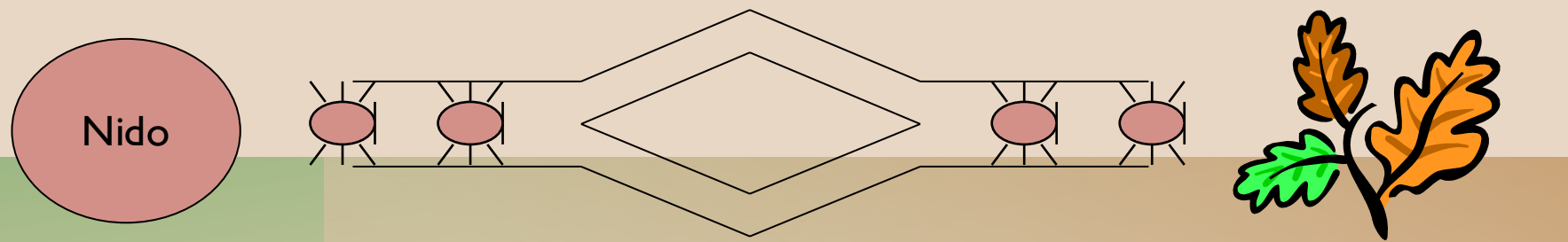
- Algoritmos evolutivos
- Optimización por enjambre de partículas
- Evolución diferencial
- Optimización con colonias de hormigas
- Algoritmos de abejas

# SWARM INTELLIGENCE

- El comportamiento emergente ‘inteligente’ no es posible con un solo individuo.
- Es una consecuencia de las interacciones de individuos en un ambiente.
- La interacción puede ser directa (ej: a través de la visión).



■ O indirecta como en el caso de las hormigas y la *stigmergy* (la modificación del ambiente a través de las feromonas)



# ¿LO ULTIMO EN SWARM INTELLIGENCE?

- Moho mucilaginoso (*Dictyostelida* y *Acrasida*)
- Comida abundante – “moho” es una ameba
- Comida escasa – se transforma en un organismo multicelular (una babosa) capaz de desplazarse...



Moho *Physarum*

# Características de los enjambres

Movimiento rápido del grupo

Reacción ante predadores (crece espacio común)

Reacción ante obstáculos

Sin colisiones entre los individuos

Fusión y division del grupo

No existe un lider exclusivo

Diferentes especies tienen diferentes agrupaciones

# Beneficios del enjambre: ahorro de energía

- Formación en V en las aves.
- Los gansos con formación en V extienden su alcance por sobre un 70% (comparado con volar aislados)
- Cada ganso se ubica en una posición precisa dependiendo de la posición de las alas del que va adelante.
- Cada ganso individual puede volar un 24% más rápido que el conjunto (sin embargo se quedan con el grupo).

# PUNTOS DE INTERÉS

- ¿Cómo nos puede llevar un enjambre a una solución cercana al optimal?
- Cuando se deja alimento a los pájaros, a menudo en minutos u horas hay un número de pájaros en esa fuente de comida (similar a la solución optimal).
- Por lo tanto podemos definir una 'función de fitness' que represente la calidad de una solución.
- El conocimiento de los individuos se puede incorporar en el algoritmo.

# **OPTIMIZACIÓN POR ENJAMBRE DE PARTICULAS**

.

# ALGUNOS PUNTOS CLAVE

- **Creación:** Kennedy and Eberhardt in 1995.
- **Partícula** – cada partícula es una solución.
  - No hay masa ni volumen
  - PERO, considera velocidad y aceleración
- **Enjambre** – no considera ajustes de velocidad (como los gansos).
- **Optimización** – descubre soluciones cercanas a un óptimo a través de una población de individuos.

# ALGORITMO BÁSICO

Crear e inicializar  $n$  partículas (soluciones) random en el espacio de búsqueda

**For** Cada Paso

**For** Cada Individuo

Actualizar la posición de la partícula incorporando velocidad a la posición actual de la partícula.

**Next** Individuo

**Next** Paso

# ACTUALIZACIÓN DE LA POSICIÓN

- ¿Cómo se determina la nueva posición de la partícula?
- Basado en su velocidad; por ejemplo:

$$x_{ij}[t+1] = x_{ij}[t] + f(v_{ij}[t+1])$$

Donde  $x$  es la posición de la partícula  $i$  en la dimensión (variable)  $j$

# ACTUALIZACIÓN DE LA VELOCIDAD

- ¿Cómo determinamos la velocidad de una partícula?
- Usando esta función:

$$v_{ij}[t+1] = v_{ij}[t] \\ + c1 * rand() * (pbestx_{ij} - presentx_{ij}) \\ + c2 * rand() * (gbestx_{ij} - presentx_{ij})$$

- Donde **c1** y **c2** son constantes y **rand()** una función que retorna un número entre 0 y 1.
- Como antes **i** es el individuo y **j** es la dimensión.
- **pbestx** es aquella partícula con *mejor* posición previa
- **gbestx** es la mejor posición previa de la población
- ¡No es tan complicado como parece!

# ACTUALIZACIÓN DE LA VELOCIDAD

- Se descompone la formula en partes:

- $v_{ij}[t+1] = v_{ij}[t]$

$$+c1*rand()*(pbestx_{ij}-presentx_{ij})$$

$$+c2*rand()*(gbestx_{ij}-presentx_{ij})$$

- La nueva velocidad de la partícula  $i$  en la dimensión  $j$  es igual a la velocidad antigua más...
- una ponderación random de la mejor posición encontrada previamente para el individuo  $i$  menos la posición actual, más...
- una ponderación random de la mejor posición encontrada previamente para la población total, menos la posición actual.

# ACTUALIZACIÓN DE LA VELOCIDAD

- ¿Por qué `rand()`?
  - Como con los AGs, para evitar la convergencia muy rápida del enjambre.
- ¿Por qué `c1` y `c2`?
  - Estas constantes pueden usarse para cambiar las ponderaciones de los individuos y de la población.
- ¿Por qué `pbest`?
  - Este es el componente que permite que los individuos vuelvan a su situación anterior (si es mejor).
- ¿por qué `gbest`?
  - Este componente es necesario para que los individuos se comparen ellos mismos con otros en sus grupos .

# PSO Y COMPORTAMIENTO SOCIAL

- Kennedy y Eberhardt discuten PSO en el contexto de comportamiento social humano.
- El parámetro  $p_{best}$  es el componente cognitivo:
  - Es decir, cuán bien lo hace un individuo basado en el rendimiento del pasado.
- El parámetro  $g_{best}$  es el componente social:
  - Cuán bien lo hace un individuo comparado con el comportamiento de otros individuos.

# VARIACIONES – PSO BASADA EN VECINDAD

- Similar a la anterior
- Se calcula *lbest* en vez de *gbest* donde *lbest* es la mejor solución en la vecindad actual
- Las vecindades normalmente no se definen como cercanas a variables de decisión
  - *Las buenas soluciones se distribuyen a través de la población*
- ¿*gbest* v *lbest*?
  - *gbest* tiende a una convergencia rápida, ya que cada partícula tiene acceso al mejor individuo
  - *lbest* tiene mayor diversidad por lo que escapa mejor de mínimos locales

# VARIABLES PSO

- Tamaño del enjambre.
  - Análogo al tamaño de la población en AGs
- Tamaño de la vecindad.
- Número de iteraciones.
- Coeficientes de aceleración ( $c_1$  y  $c_2$ ).
  - Manejan la relación entre  $p_{best}$  y  $g_{best}/l_{best}$
  - Valores bajos pueden implicar progreso lento
  - Valores altos pueden implicar que el algoritmo pierda buenas soluciones

# CRITERIO DE TÉRMINO

- Máximo de Iteraciones
- Se encuentra una solución aceptable
- No hay mejoras después de un cierto número de iteraciones
- ...

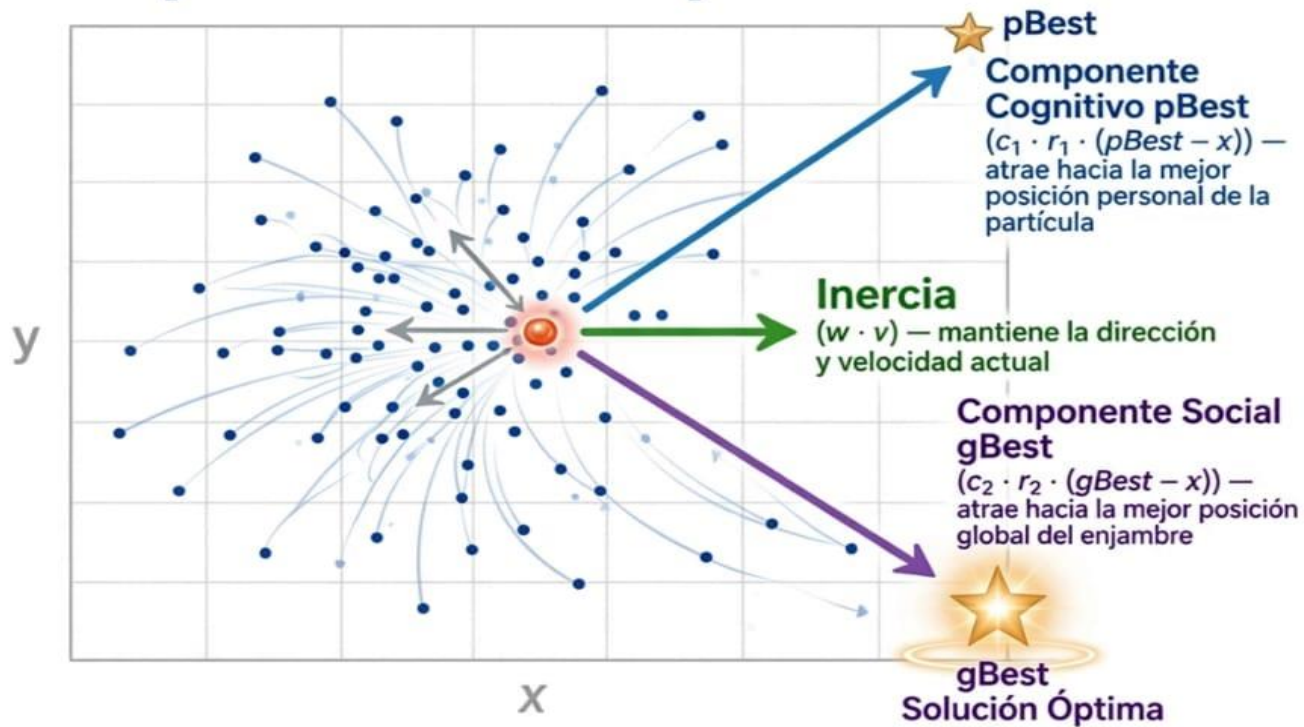
# PSO APLICACIONES

- Diseño:
  - Antenas, alas de aeronaves, amplificadores, controladores, circuitos etc...
- Scheduling y planificación:
  - Planificación operacional, asignación de tareas, TSP

# OPTIMIZACIÓN POR ENJAMBRE DE PARTÍCULAS — PSO

Algoritmo inspirado en el comportamiento colectivo de enjambres para encontrar el óptimo global en un espacio de búsqueda

## Espacio de Búsqueda 2D



## Ecuación de Velocidad

$$v_{nueva} = w \cdot v + c_1 \cdot r_1 \cdot (pBest - x) + c_2 \cdot r_2 \cdot (gBest - x)$$

- **Inercia ( $w \cdot v$ ):** mantiene la velocidad anterior, favoreciendo que la partícula continúe en su dirección actual
- **Componente Cognitivo pBest:** impulsa a la partícula hacia su mejor posición personal encontrada hasta ahora
- **Componente Social gBest:** impulsa a la partícula hacia la mejor posición global encontrada por todo el enjambre

## Actualización de Posición

$$x_{nueva} = x + v_{nueva}$$

## Cómo funciona PSO — Pasos clave:

### 1 Inicialización

Se generan varias partículas con posición ( $x$ ) y velocidad ( $v$ ) aleatorias

### 2 Evaluación

Cada partícula evalúa la función objetivo y guarda su mejor posición personal pBest

### 3 Actualización

Se actualiza la velocidad usando la ecuación y luego la posición de cada partícula

### 4 Iteración

Se repite el proceso hasta alcanzar el óptimo gBest o máximo de iteraciones

Leyenda: • = Partícula | pBest = Mejor posición personal | gBest = Mejor posición global |  $w$  = peso de inercia,  $c_1, c_2$  = coeficientes de aceleración, |  $r_1, r_2$  = valores aleatorios en  $[0,1]$

# MENTE COLMENA

- Un sistema donde muchos individuos actúan como si fueran una sola mente.
- Ejemplos en la naturaleza
  - Hormigas y abejas
  - Pájaros en bandadas
  - Redes de hongos
- Ejemplos en la sociedad
  - Redes e Internet
- Ejemplos en ciencia ficción
  - Los Borg (Star Trek)



# PROBLEMAS

- Los sistemas de enjambre son difíciles de ‘programar’ ya que los problemas son a menudo difíciles de definir.
  - Las soluciones son emergentes en los sistemas.
  - Las soluciones son el resultado del comportamiento y las interacciones entre agentes individuales.

# SOLUCIONES POSIBLES PARA CREAR SISTEMAS CON INTELIGENCIA DE ENJAMBRE

- Crear un catálogo de comportamientos colectivos 😊
- Modelar cómo insectos sociales realizan tareas colectivamente.
  - Usar el modelo como base para desarrollar alternativas de uso de la IA.
  - Los parámetros de un modelo pueden ser “ajustados” dentro de un “rango biológico relevante” o incluso agregando factores no biológicos al modelo.

# CARACTERÍSTICAS DE SWARM INTELLIGENCE

- Swarm Intelligence es la interacción local de muchos agentes simples para alcanzar una meta global.
- Emergencia
  - El comportamiento global único es consecuencia (surge de) la interacción entre muchos agentes.
- Stigmergy
  - Comunicación indirecta.
    - Generalmente a través del ambiente.

# PROPIEDADES DE SWARM INTELLIGENCE

- Agentes deben ser simples.
- Comunicación indirecta entre agentes.
- El comportamiento global resulta ser emergente.
  - No se necesita una programación local específica.
- Comportamientos son robustos.
  - Se requiere en el caso de ambientes impredecibles.
- Los individuos no son importantes.

# INGREDIENTES DE AUTOORGANIZACION

- Feedback Positivo.
- Feedback Negativo.
- Amplificación de Fluctuaciones – aleatoriedad.
- Dependencia de múltiples interacciones.

# FEEDBACK POSITIVO

El feedback positivo refuerza las buenas soluciones.

- Las hormigas son capaces de atraer más ayuda cuando encuentran una fuente de alimentos.
- A mayor cantidad de hormigas en un camino, aumenta el nivel de feromona y atrae aún más hormigas.

# FEEDBACK NEGATIVO

El feedback negativo elimina las soluciones malas (o antiguas) de la memoria colectiva.

- Decaimiento de la feromona.
- Las fuentes de alimento distantes son las últimas en ser exploradas.
  - La feromona tiene menos tiempo para decaer en soluciones cercanas.

# ALEATORIEDAD

La aleatoriedad permite el surgimiento de nuevas soluciones.

- Las decisiones de las hormigas son aleatorias.
  - Probabilidad de exploración.
- Las fuentes de comida son encontradas aleatoriamente.

# INTERACCIONES MULTIPLES

Solamente a través de las interacciones de muchos individuos es posible encontrar una solución.

- Una sola hormiga no puede buscar comida. **WHY?**
  - La feromona decae muy rápido.
- Se requieren muchas hormigas para mantener un rastro de feromona.
- Más comida puede encontrarse más rápidamente.

# EN RESUMEN (SWARM INTELLIGENCE)

- Es adecuado para encontrar soluciones que no requieren un control preciso respecto de cómo se alcanza el objetivo.
- Necesita un número grande de agentes.
- Los agentes pueden ser muy simples.
- Los comportamientos son robustos.

# PROPIEDADES DE LA AUTO-ORGANIZACIÓN

- Creación de estructuras.
  - Nidos, búsqueda de caminos, u organización social.
- Cambios resultan de la existencia de múltiples caminos de desarrollo.
  - Fases coordinadas y no coordinadas.
- Posible coexistencia de múltiples estados estables.
  - Dos fuentes de alimentos iguales.

# TIPOS DE INTERACCION PARA INSECTOS SOCIALES

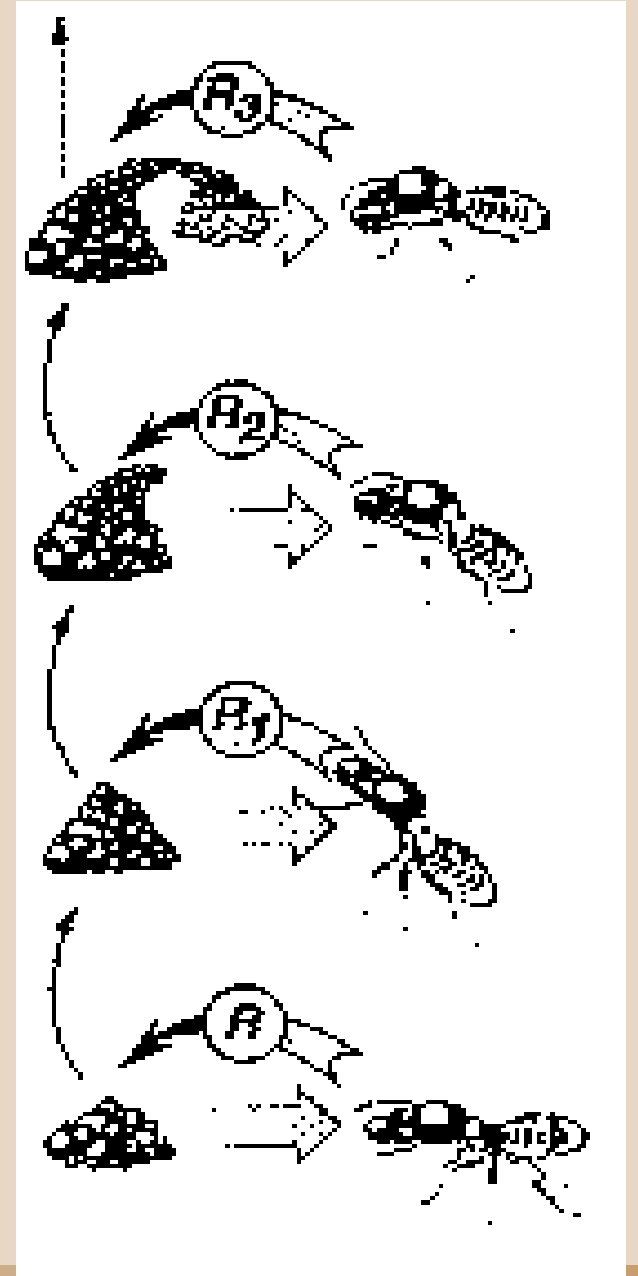
- Interacciones Directas.
  - Intercambio de alimentos/líquidos, contacto visual, contacto químico (feromonas).
- Interacciones Indirectas (Stigmergy).
  - El comportamiento individual modifica el ambiente, el que a su vez modifica el comportamiento de otros individuos.

# STIGMERGY

- Es un mecanismo de coordinación indirecta, a través del ambiente, entre agentes o acciones.
- El principio de este concepto quiere decir que los rastros dejados en el ambiente por una acción individual, estimulan la realización de una acción exitosa, por el mismo agente, o uno diferente.
- Agentes que responden a los rastros en el ambiente refuerzan esos comportamientos.

# EJEMPLO DE STIGMERGY

- Construcción de pilares por termitas



# HORMIGAS, UNA MIRADA MAS CERCANA

- Ejemplos de insectos en el mundo real
- Teoría de Swarm Intelligence
- De insectos a algoritmos IA realistas
- Ejemplos de aplicaciones IA

# INSECTOS SOCIALES

- Los beneficios de la inteligencia de enjambre para la solución de problemas incluyen como características importantes el ser:
  - Flexibles
  - Robustos
  - Descentralizados
  - Auto-Organizados

# ABEJAS

- Cooperación de la colonia
- Regulación de la temperatura de la colmena
- Eficiencia a través de la especialización: división del trabajo en la colonia
- Comunicación : Fuentes de alimentos son explotadas de acuerdo a su calidad y distancia de la colmena

# AVISPAS

- Recolectoras de pulpa, recolectoras de agua y constructoras
- Nidos complejos
  - Columnas horizontales
  - Coberturas protectoras
  - Chimenea para la entrada principal

# TERMITAS

- Paredes exteriores en forma de cono y ductos de ventilación.
- Cámaras de crianza en la colmena central.
- Respiraderos de enfriamiento en espiral.
- Pilares de apoyo.

# HORMIGAS

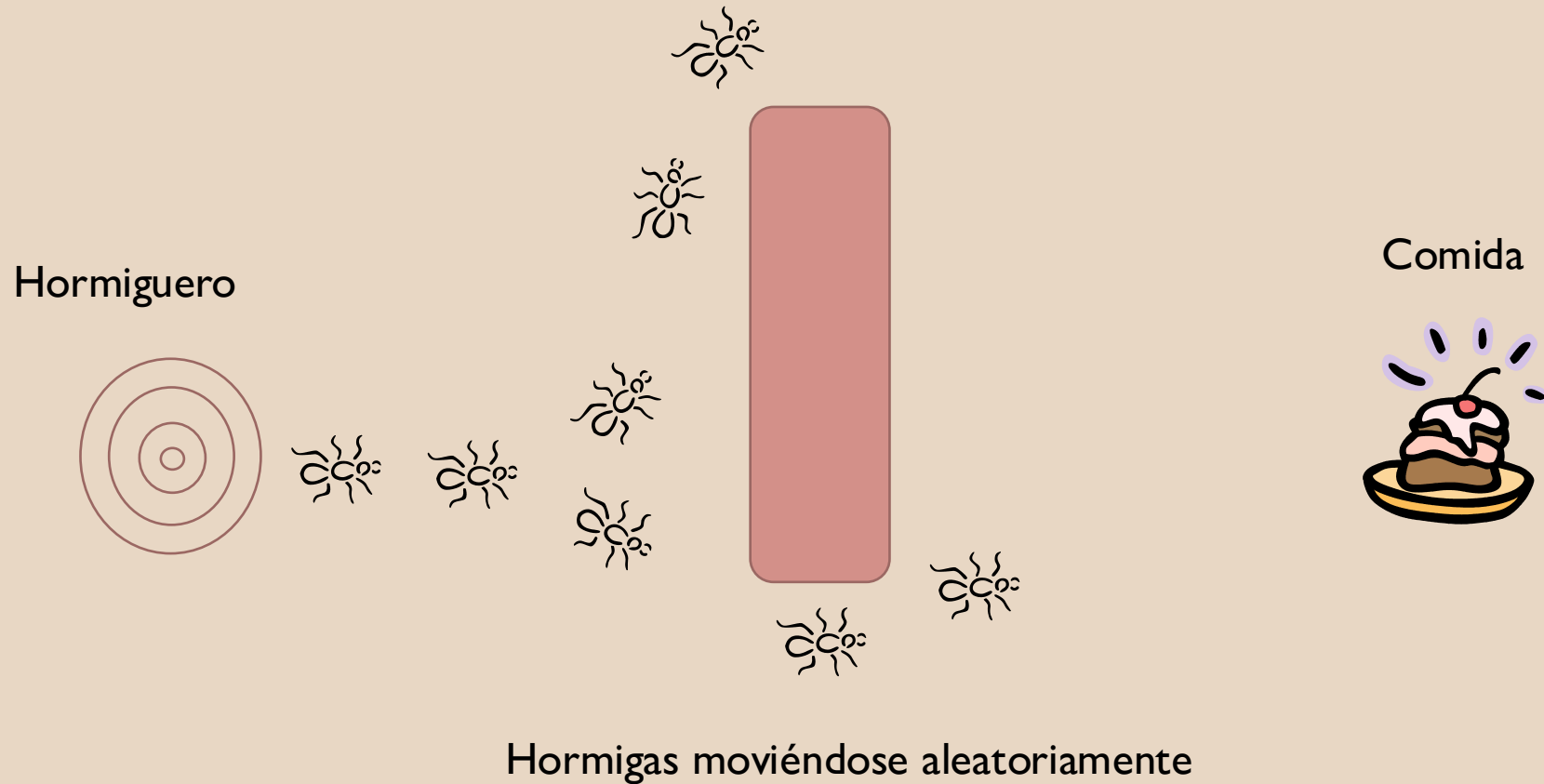
- Organización de caminos hacia y desde los sitios con alimentos dejando rastros de feromona.
- División del trabajo entre hormigas mayores y hormigas jóvenes.

# RESUMEN DE INSECTOS

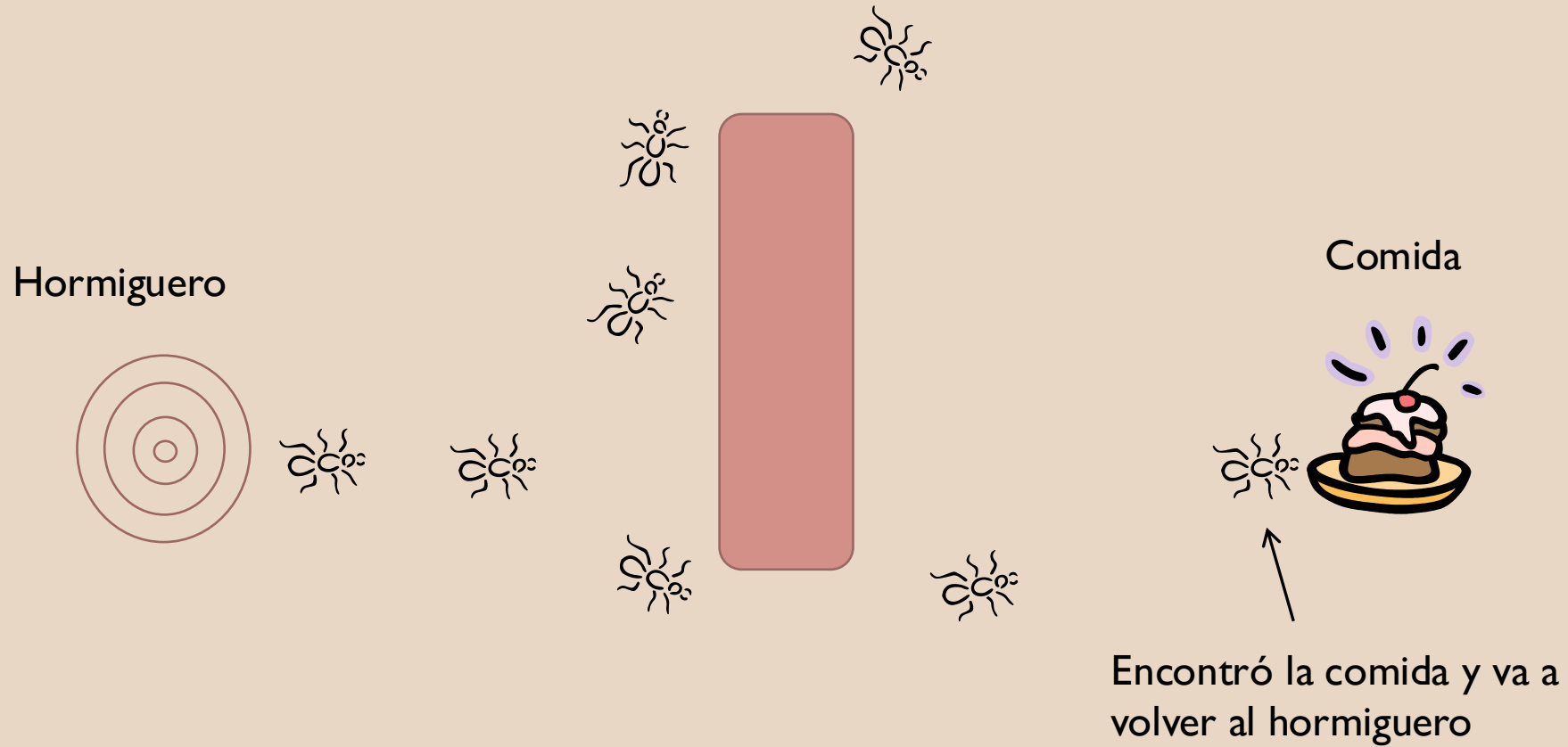
- La complejidad y sofisticación de la Auto-Organización se lleva a cabo sin un líder claro.
- Lo que aprendemos de los insectos sociales puede ser aplicado al campo del Diseño de Sistemas Inteligentes.
- El modelamiento de los insectos sociales y su Auto-Organización puede ayudar a diseñar dispositivos distribuidos para la solución de problemas. Esto también se conoce como Sistemas de Swarm Intelligence.

# UNA MIRADA AL COMPORTAMIENTO DE LAS HORMIGAS REALES

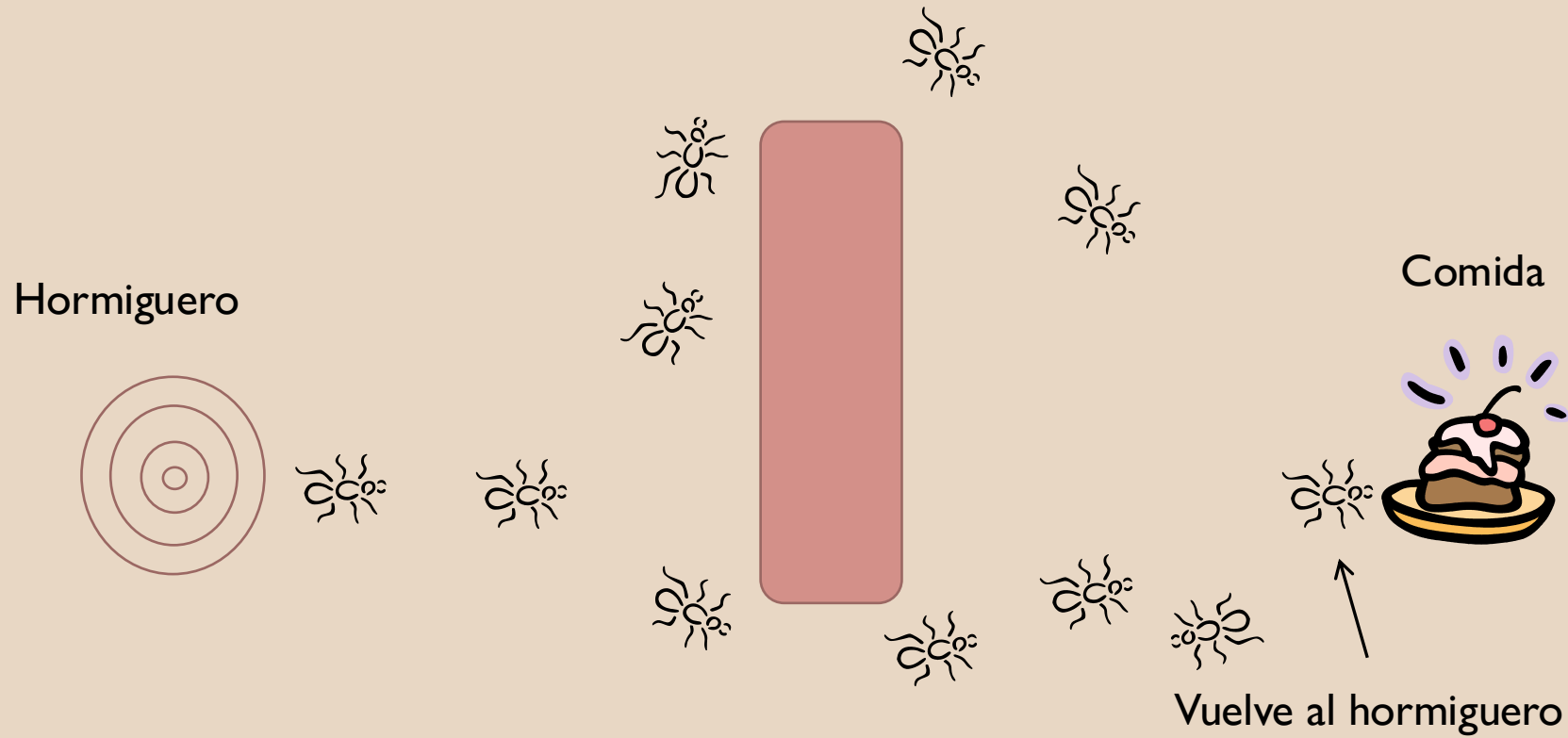
# CAMINO MÁS CORTO (HORMIGAS REALES)



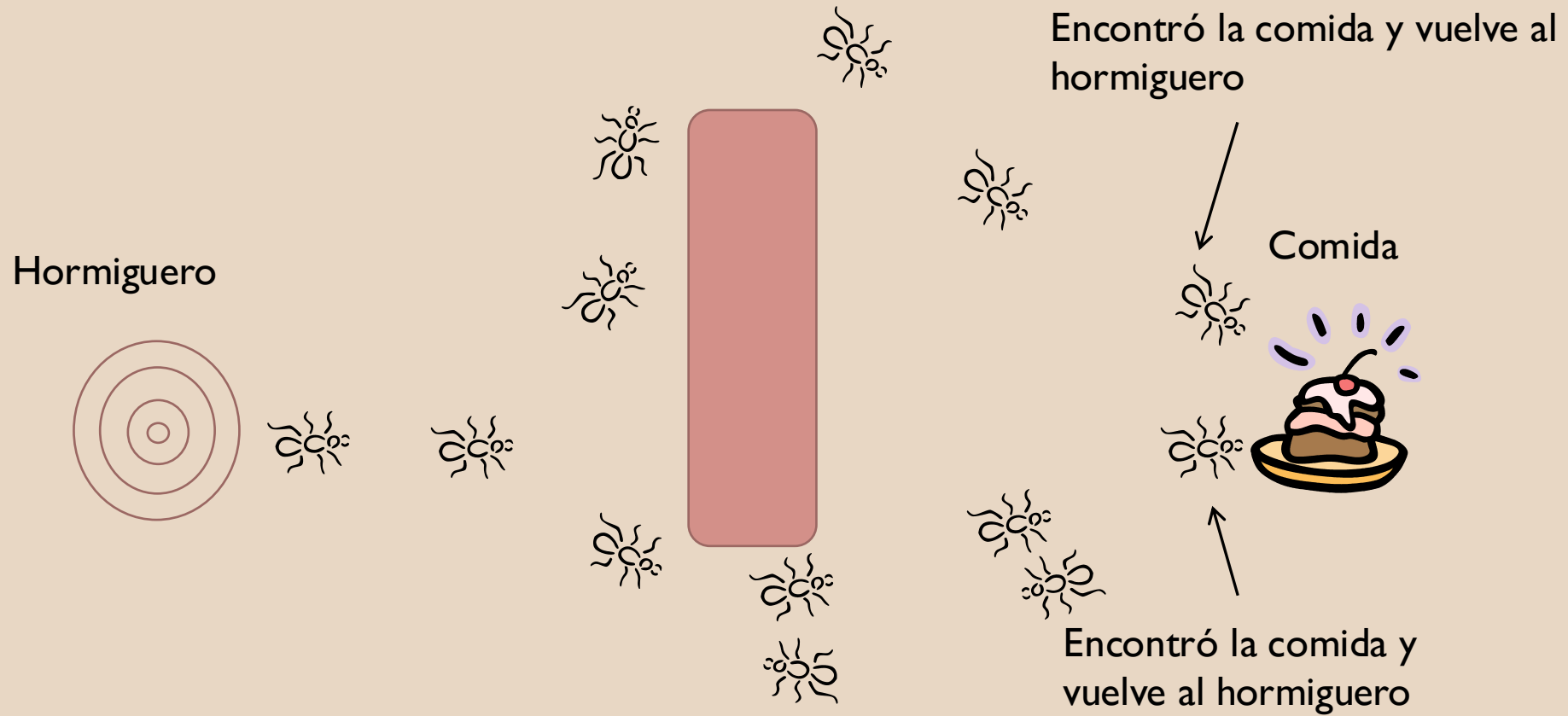
# CAMINO MÁS CORTO (HORMIGAS REALES)



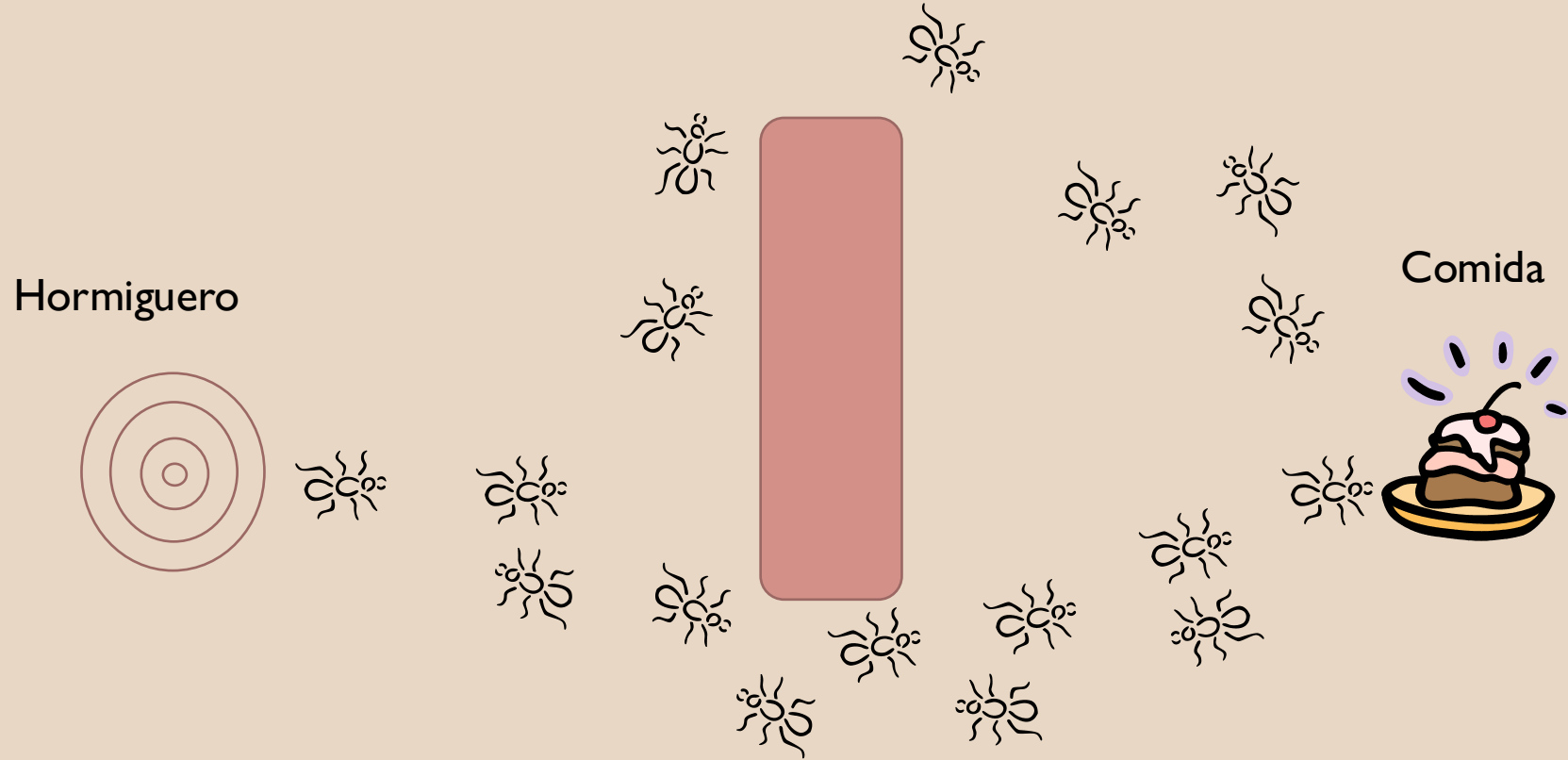
# CAMINO MÁS CORTO (HORMIGAS REALES)



# CAMINO MÁS CORTO (HORMIGAS REALES)



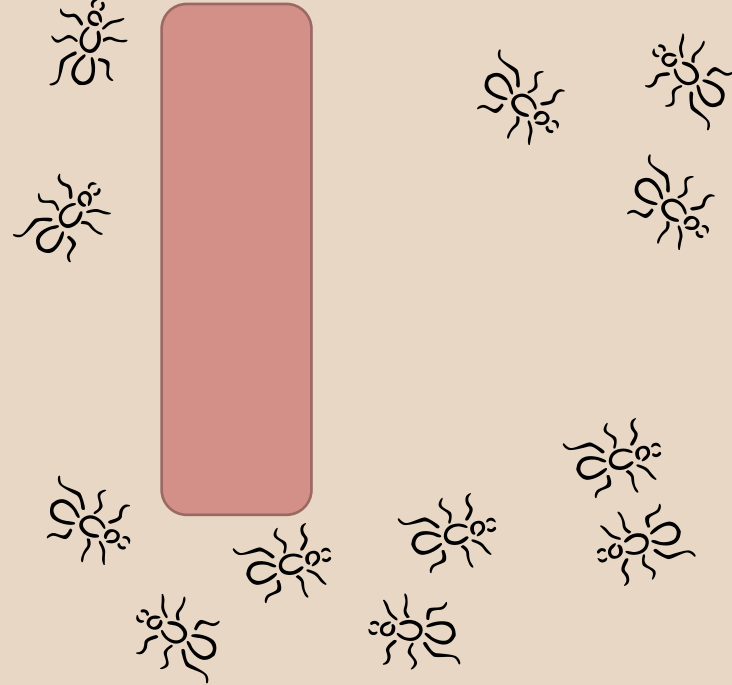
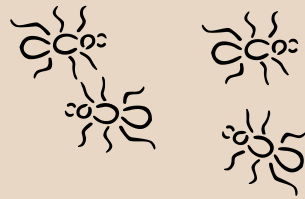
# CAMINO MÁS CORTO (HORMIGAS REALES)



# CAMINO MÁS CORTO (HORMIGAS REALES)

Tiene mayor probabilidad de ir por abajo (el camino donde pasaron más hormigas - más feromona)

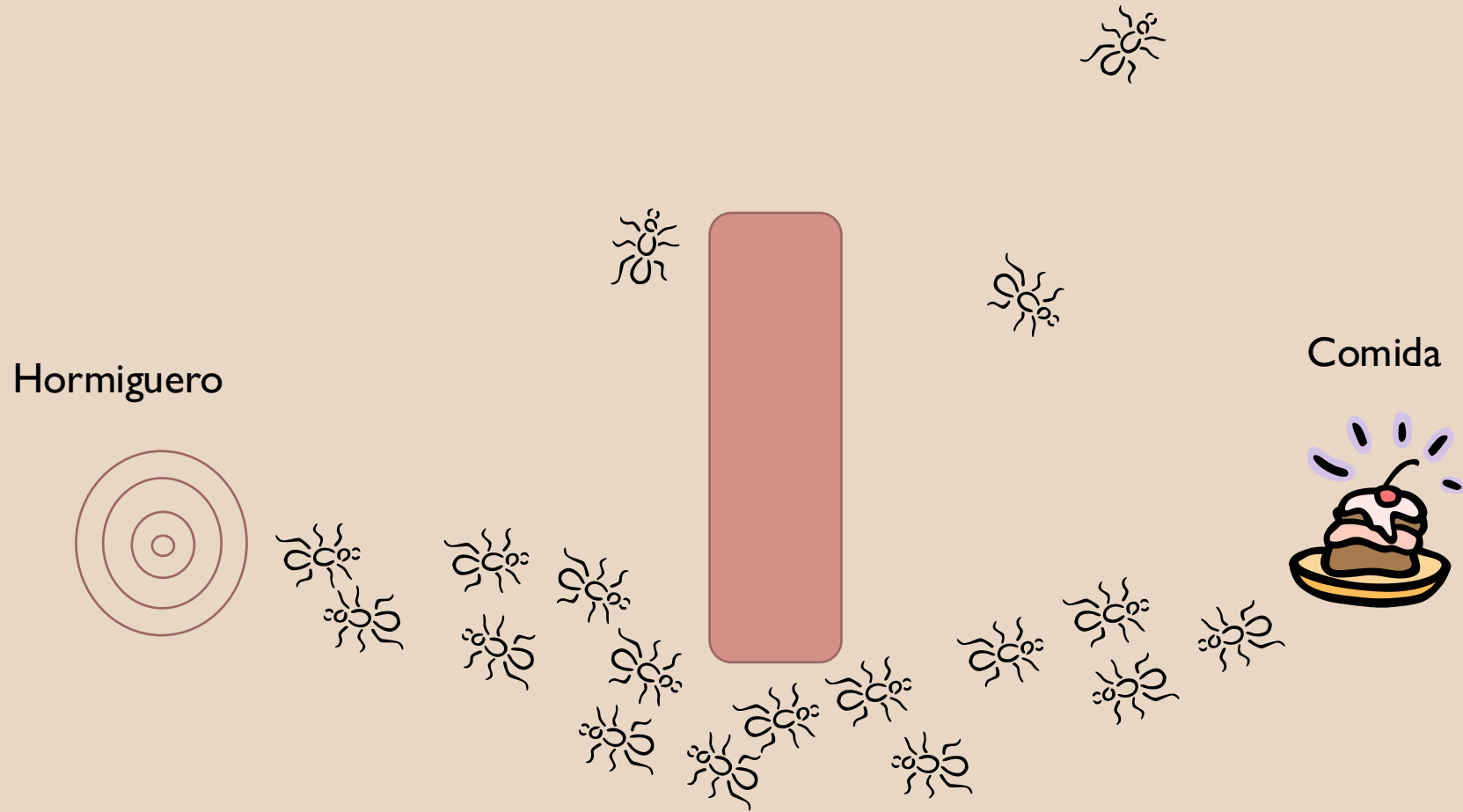
Hormiguero



Comida



# CAMINO MÁS CORTO (HORMIGAS REALES)



# HORMIGAS $\equiv$ AGENTES

- Stigmergy puede ser operacional
  - Coordinación por interacciones indirectas es más atractiva que la comunicación directa.
  - Stigmergy reduce (o elimina) las comunicaciones entre agentes.

# EL ENFOQUE ACO, MARCO GENERAL.

- El marco general en el cual situar al enfoque ACO es dentro la denominada **SWARM INTELLIGENCE** o **INTELIGENCIA COLECTIVA**.
- Este marco presupone un sistema de agentes que obedecen a un conjunto de reglas muy simples, pero que actuando cooperativamente, generan un sistema mucho más complejo.
- Ejemplos: Hormigas, termitas, abejas, bandada, etc.

# CONCEPTOS PREVIOS

Métodos básicos para explorar el espacio de búsqueda a partir de los cuales se derivan las metaheurísticas:

- *Búsqueda Local*
- *Algoritmos constructivos*

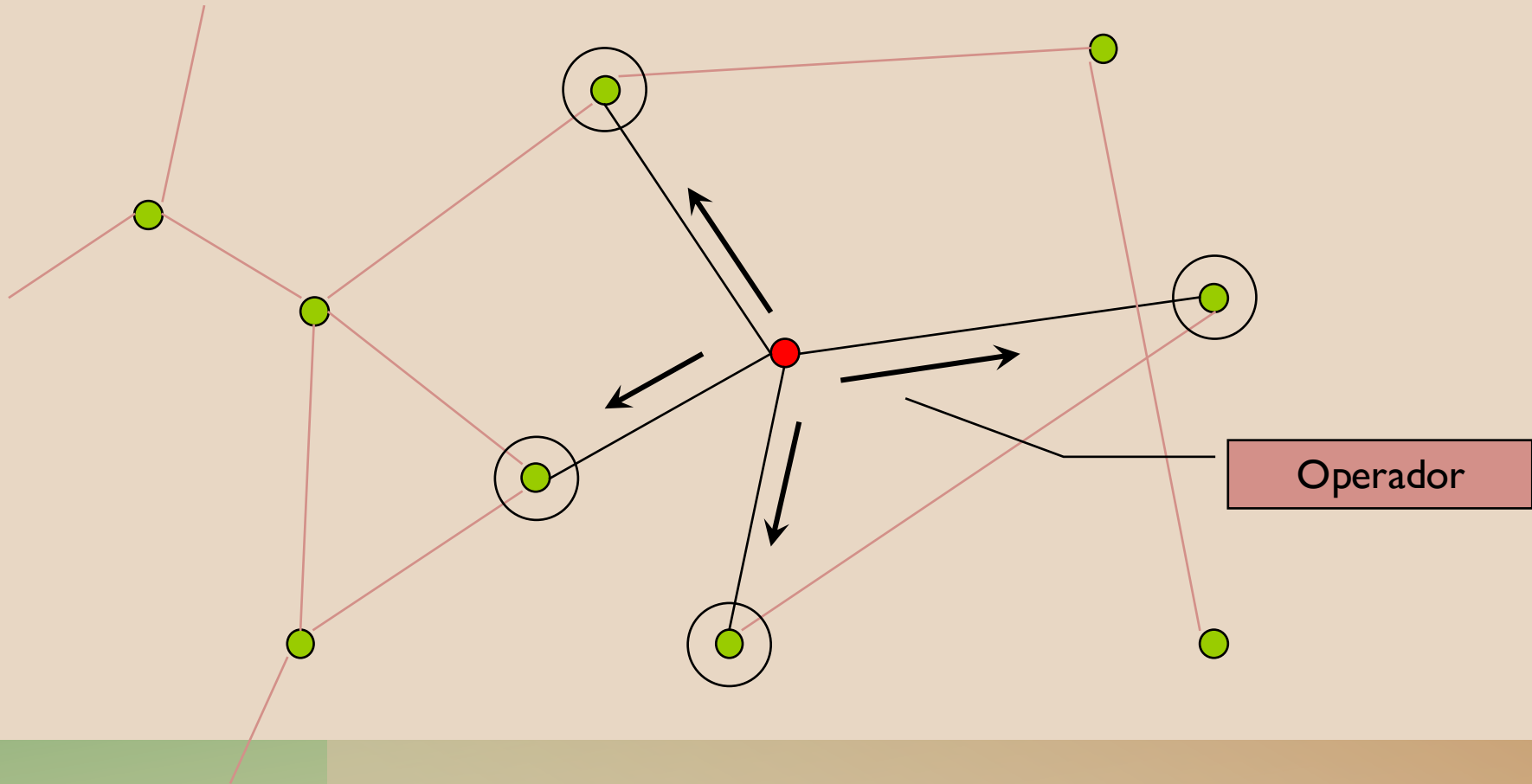
# CONCEPTOS PREVIOS (CONT.)

## *Búsqueda Local*

- Presupone una estructura bien definida del espacio de búsqueda (uso del concepto de vecindario).
- Comienza desde una solución inicial y repetidamente trata de mejorarla a través de cambios locales.
- Cada cambio realizado le permite al método, “moverse” hacia otros puntos del espacio de búsqueda dentro del vecindario.

# CONCEPTOS PREVIOS (CONT.)

## VECINDARIO DE UN PUNTO EN EL ESPACIO



# CONCEPTOS PREVIOS (CONT.)

## *Algoritmos constructivos*

- Las soluciones son construidas iterativamente añadiendo componentes a una solución partiendo desde una solución ‘vacía’.
- Por ejemplo, en el problema del Vendedor Viajero (TSP) la solución es construida añadiendo una ciudad tras otra incrementando la longitud del tour.

# DE INSECTOS A ALGORITMOS DE IA REALISTAS

# DE HORMIGAS A ALGORITMOS

- La información en la inteligencia de enjambre nos permite modelar:
  - Solución de problemas.
  - Algoritmos.
  - Aplicaciones del mundo real.

# MODELAMIENTO

- Observar el fenómeno.
- Crear un modelo con inspiración biológica.
- Explorar el modelo sin restricciones.

# MODELAMIENTO (CONTINUACIÓN)

- Crea una version simplificada de la realidad.
- Los elementos cuantitativos observables se transforman en variables del modelo.
- Otras variables (no visibles de manera destacada) constituyen conexiones (relaciones).

# UN BUEN MODELO TIENE...

- Parsimonia (simplicidad)
- Coherencia
- Refutabilidad
- Los valores de los parámetros corresponden a valores de sus contrapartes naturales

# PROBLEMA DEL VENDEDOR VIAJERO

Inicializar

**Loop** /\* a este nivel cada loop es una iteración \*/

Cada hormiga se posiciona en un nodo de partida.

**Loop** /\* a este nivel cada loop es un paso \*/

Cada hormiga aplica una regla de transición de estados, para construir incrementalmente una solución, y una regla de actualización de la feromona.

**Hasta** que todas las hormigas hayan construido una solución.

Se aplica, globalmente, una regla de actualización de la feromona global.

**Hasta** Condición de término.

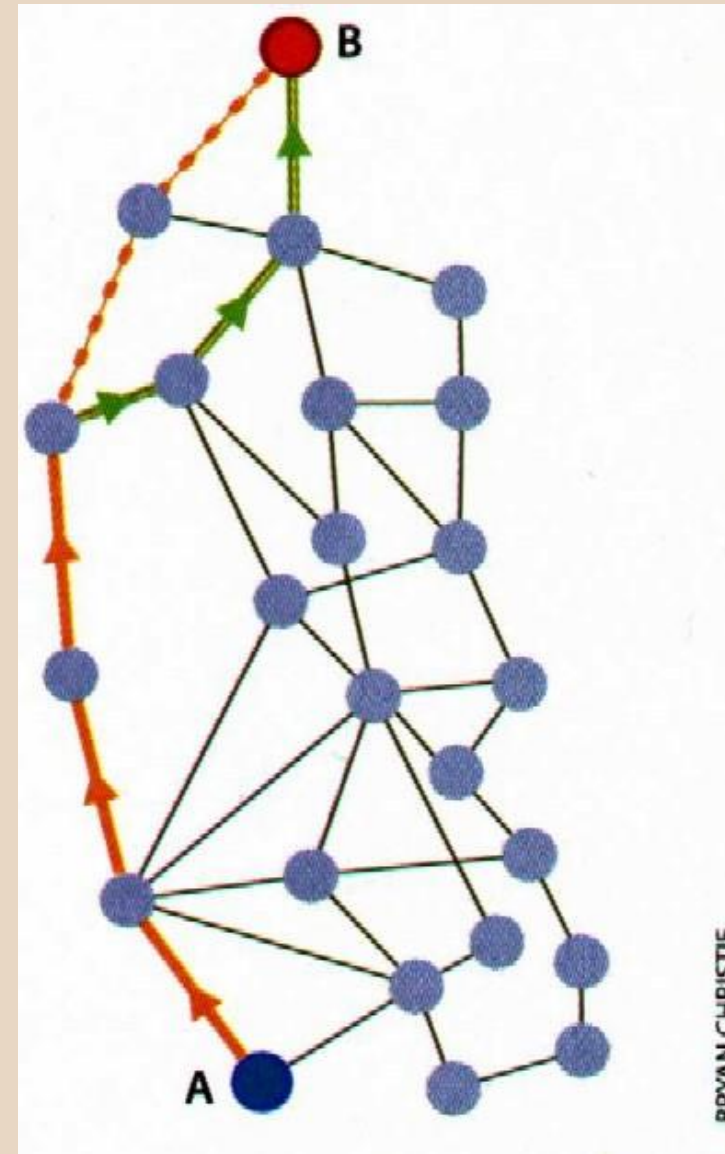
BIENVENIDOS AL MUNDO REAL

# REDES DE COMUNICACIONES

- Ruteamiento de paquetes a su destino en el tiempo más corto.
- Similar al camino mínimo.
- Se mantienen estadísticas sobre rutas previas (aprendizaje a partir de la experiencia).

# ¿Waze?

- Ruta más corta
- Congestión
- Adaptabilidad
- Flexibilidad



# ANT SYSTEMS

- Hasta 1992 los sistemas de colonias de hormigas eran usados exclusivamente para analizar el comportamiento de las hormigas reales.



# ALGORITMOS DE HORMIGAS

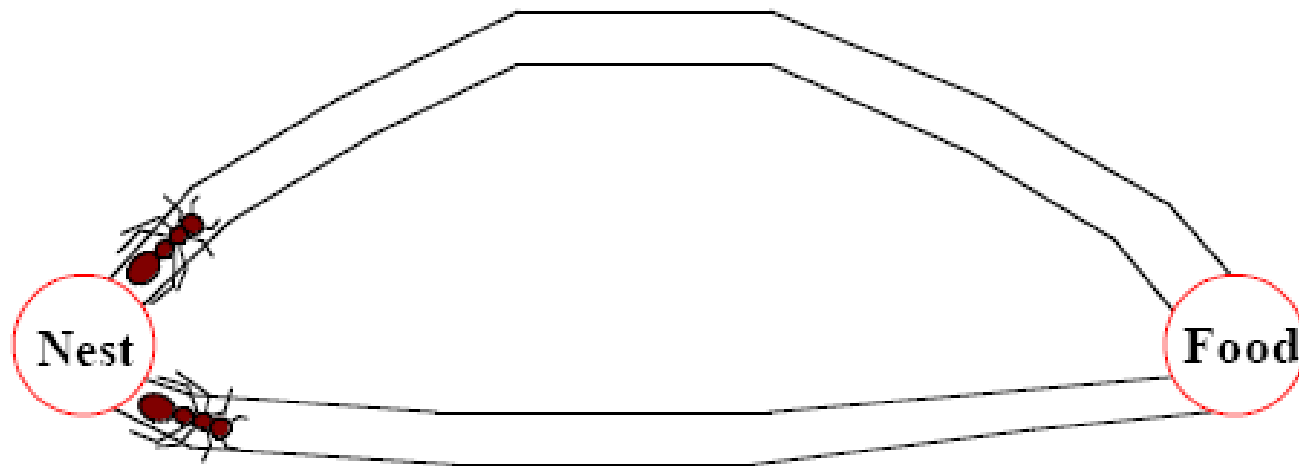
- Las hormigas son prácticamente ciegas, pero se las arreglan para encontrar los caminos a las fuentes de alimentación. ¿Cómo lo hacen?
- Estas observaciones inspiraron un nuevo tipo de algoritmos llamados *ant algorithms* (o ant systems).

# ALGORITMOS DE HORMIGAS

- Son una aproximación basada en una población. En ese sentido son similares a los AGs.
- Existe una población de hormigas, donde cada hormiga busca una solución y luego se comunica con las otras hormigas.

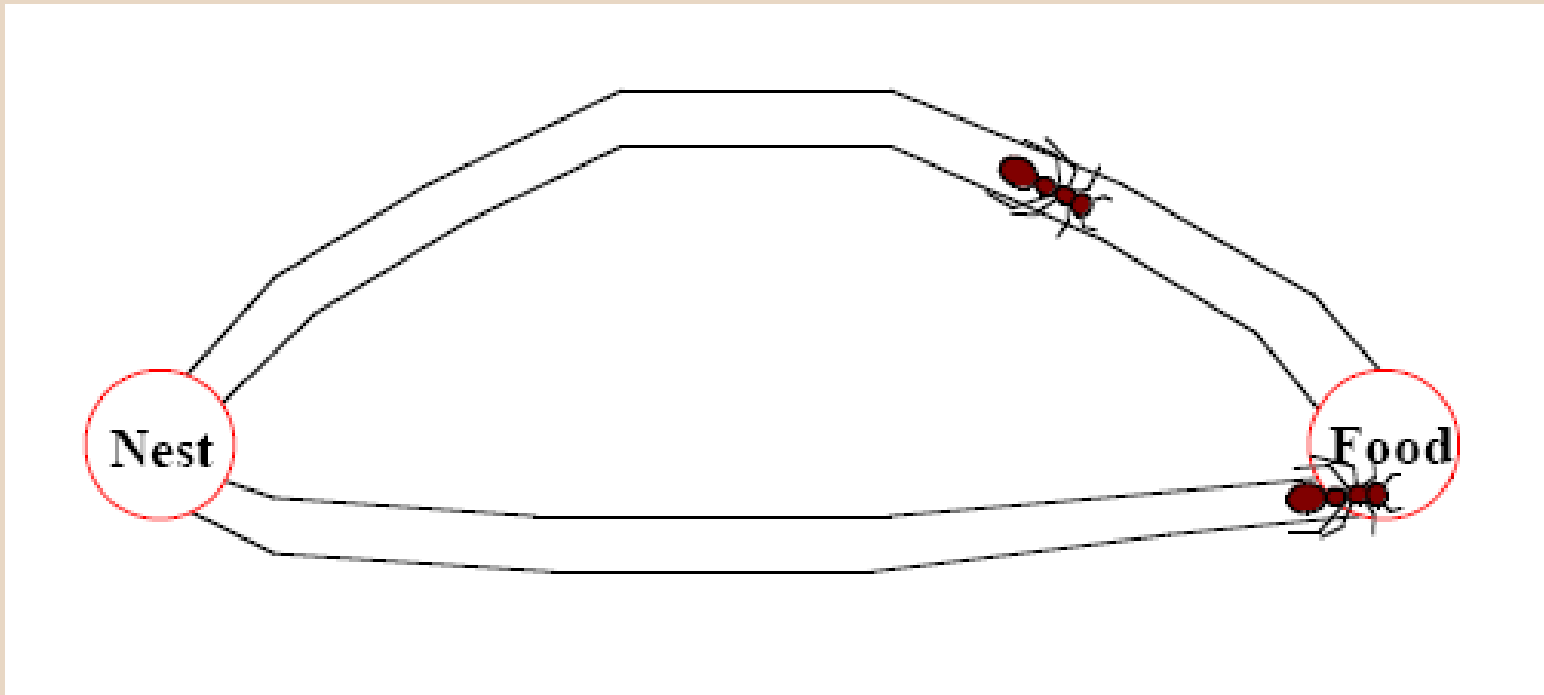
# BÚSQUEDA DE ALIMENTOS

- Búsqueda cooperativa usando rastros de feromona



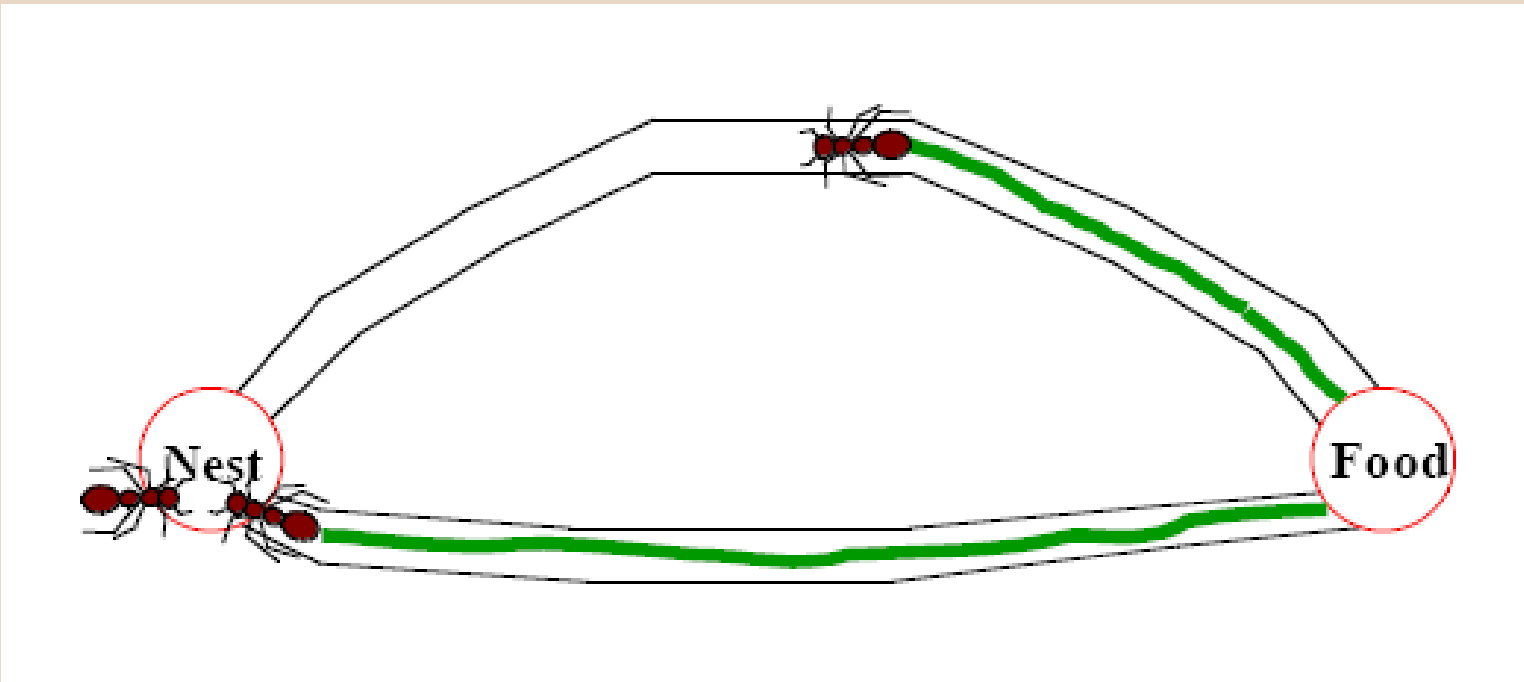
# BÚSQUEDA DE ALIMENTOS

- Búsqueda cooperativa usando rastros de feromona



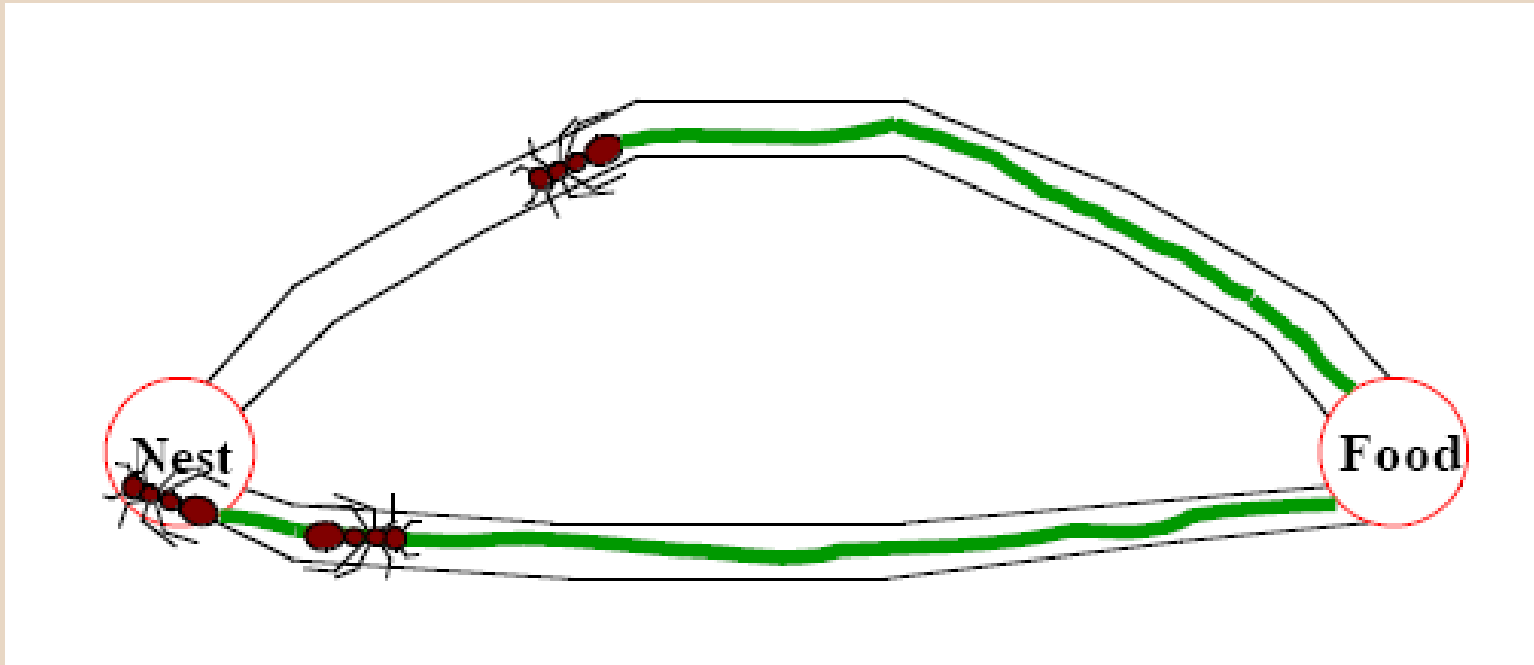
# BÚSQUEDA DE ALIMENTOS

- Búsqueda cooperativa usando rastros de feromona



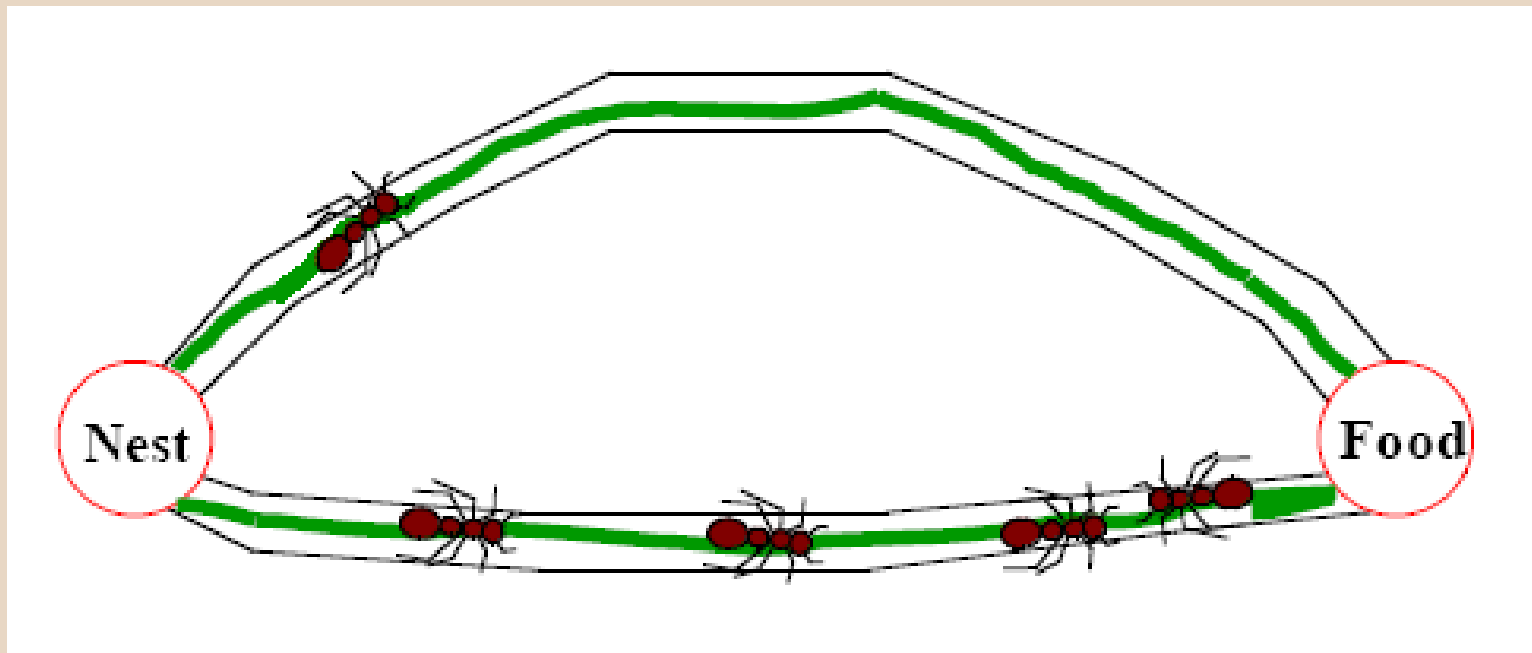
# BÚSQUEDA DE ALIMENTOS

- Búsqueda cooperativa usando rastros de feromona



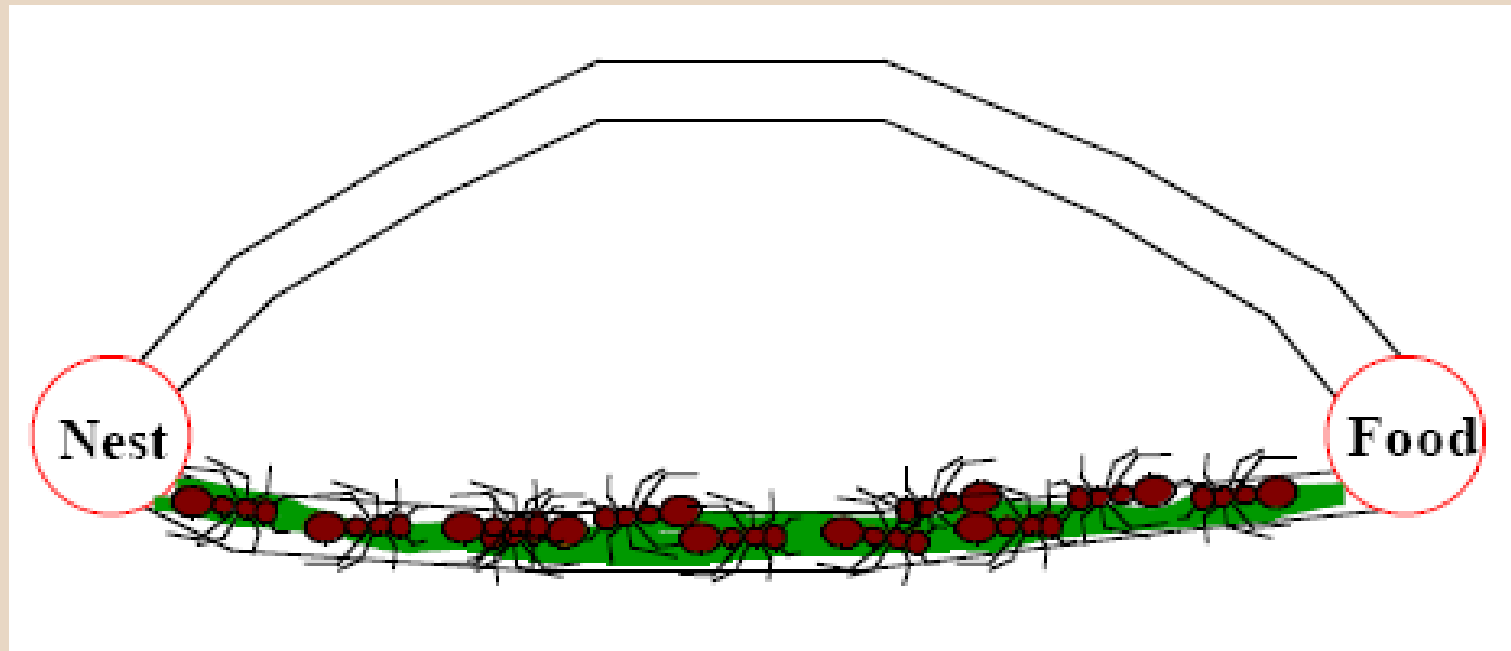
# BÚSQUEDA DE ALIMENTOS

- Búsqueda cooperativa usando rastros de feromona



# BÚSQUEDA DE ALIMENTOS

- Búsqueda cooperativa usando rastros de feromona



# ALGORITMOS DE HORMIGAS

Tiempo,  $t$ , es discreto.

En cada unidad de tiempo, una hormiga se mueve una distancia,  $d$ .

Una vez que la hormiga se mueve, deposita en el camino una unidad de feromona.

En  $t=0$ , no hay feromona en los arcos.

# ALGORITMOS DE HORMIGAS

- Estamos interesados en explorar el espacio de búsqueda, no simplemente encontrar una ruta.
- Debemos permitir que las hormigas exploren caminos y sigan los mejores caminos con alguna *probabilidad* proporcional a la intensidad del rastro de feromona.
- No esperamos que sigan simplemente una ruta con la mayor cantidad de feromona, ya que podríamos estancarnos en una ruta sub-óptima.

# ALGORITMOS DE HORMIGAS

- La probabilidad de que una hormiga siga cierta ruta es una función, no solamente de la intensidad de feromona, sino también de una función que refleja lo que la hormiga ve (*visibilidad*).
- El rastro de feromona no es eterno..., se necesita un proceso de evaporación.

# ALGORITMOS DE HORMIGAS Y EL TSP

- Al inicio del algoritmo se coloca una hormiga en cada ciudad.
- Variaciones de esta estrategia fueron chequeadas por Dorigo.

# ALGORITMOS DE HORMIGAS Y EL TSP

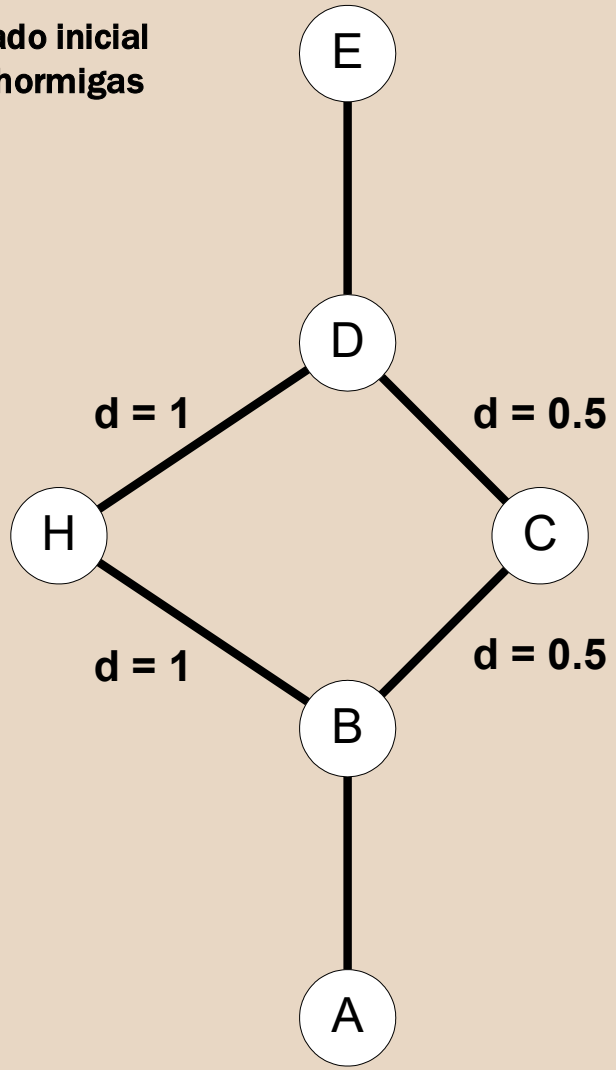
- El tiempo,  $t$ , es discreto.  $t(0)$  marca el inicio del algoritmo. En  $t+1$  cada hormiga se habrá desplazado a una nueva ciudad.
- Suponiendo que el TSP se representa por un grafo totalmente conexo, cada arco tiene una *intensidad del rastro*. Esto representa el rastro de feromona dejado por las hormigas.
- Se denota por  $T_{i,j}(t)$  a la intensidad del rastro de feromona en el arco  $(i,j)$  en el tiempo  $t$ .

# ALGORITMOS DE HORMIGAS Y EL TSP

- Cuando una hormiga “decide” a qué ciudad se moverá, lo hace en base a una probabilidad considerando la distancia a esa ciudad y la intensidad de feromona en el arco que las conecta.
- La distancia a la próxima ciudad se conoce como la *visibilidad*,  $n_{ij}$ , y se define como  $1/d_{ij}$ , donde  $d_{ij}$  es la distancia entre las ciudades  $i$  y  $j$ .

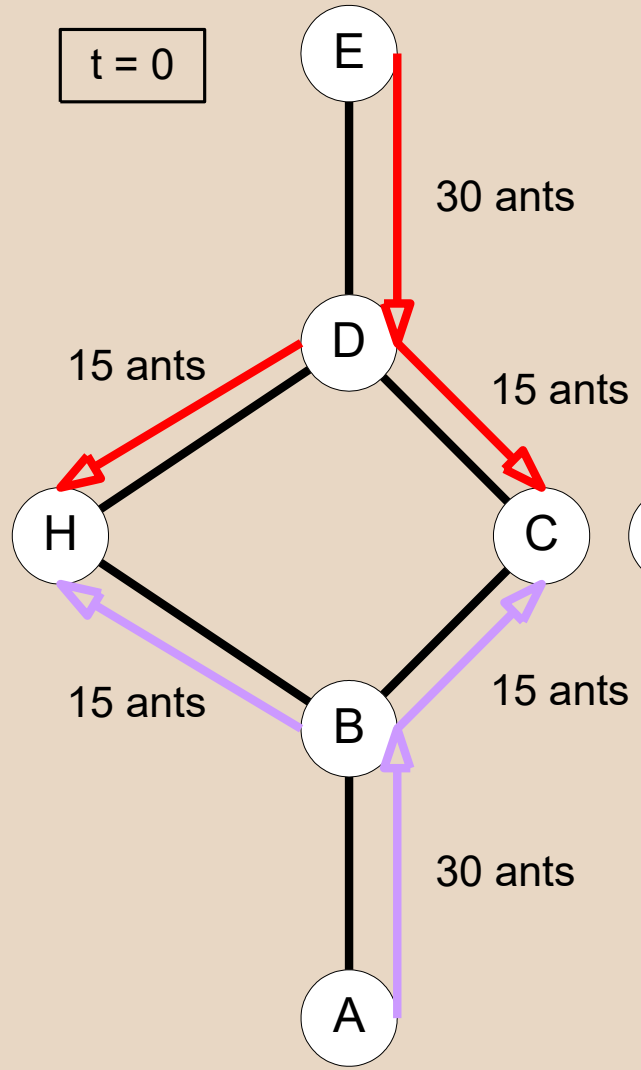
## Trazas de feromona

Estado inicial  
sin hormigas



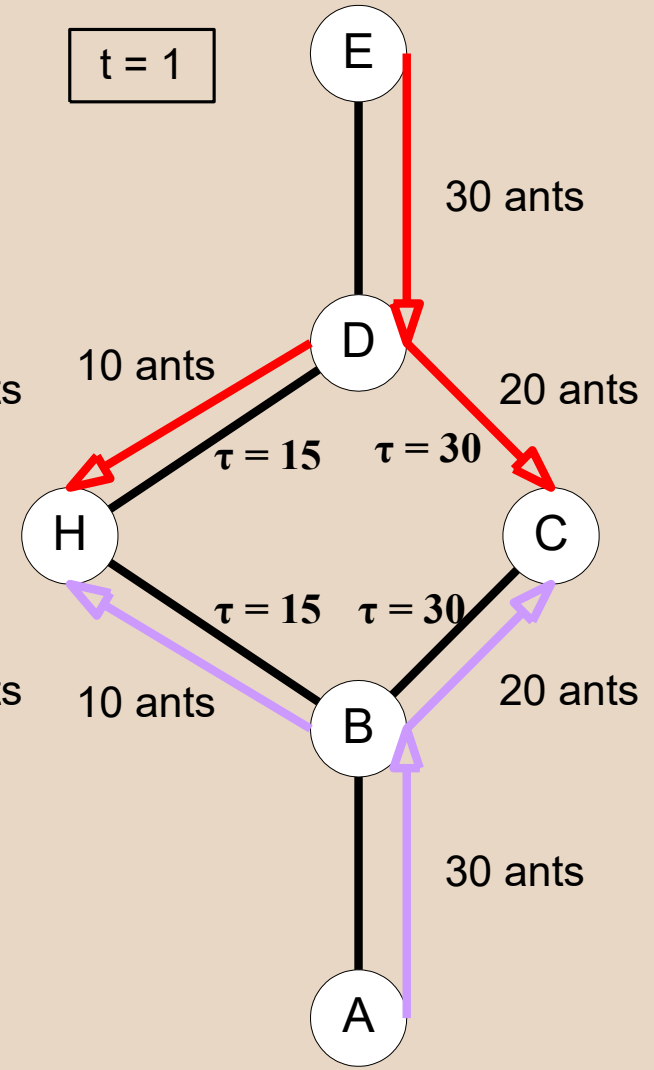
(a)

$t = 0$



(b)

$t = 1$



(c)

# LA IDEA BÁSICA

Las hormigas son *agentes* que:

Se desplazan entre los nodos en un grafo.

*Eligen* donde ir en base a la intensidad de la feromona.

El camino de una hormiga representa una candidata a solución específica.

Cuando una hormiga termina de construir una solución, se deja feromona en el camino, dependiendo de la calidad de la solución.

El rastro de feromona afecta el comportamiento de las otras hormigas por *'stigmergy'* ...

# ALGORITMOS DE HORMIGAS Y EL TSP

- En cada unidad de tiempo se produce una *evaporación*.
- La cantidad de evaporación,  $\rho$ , es un valor entre 0 y 1.

# ALGORITMOS DE HORMIGAS Y EL TSP

- Para evitar que las hormigas entren en loop, se utiliza una estructura de datos Tabú.
- Esto evita que las hormigas vuelvan a pasar por ciudades que ya han visitado.
- $Tabu_k$  se define como la lista para la  $k$ -ésima hormiga y contiene todas las ciudades que ya han sido visitadas.

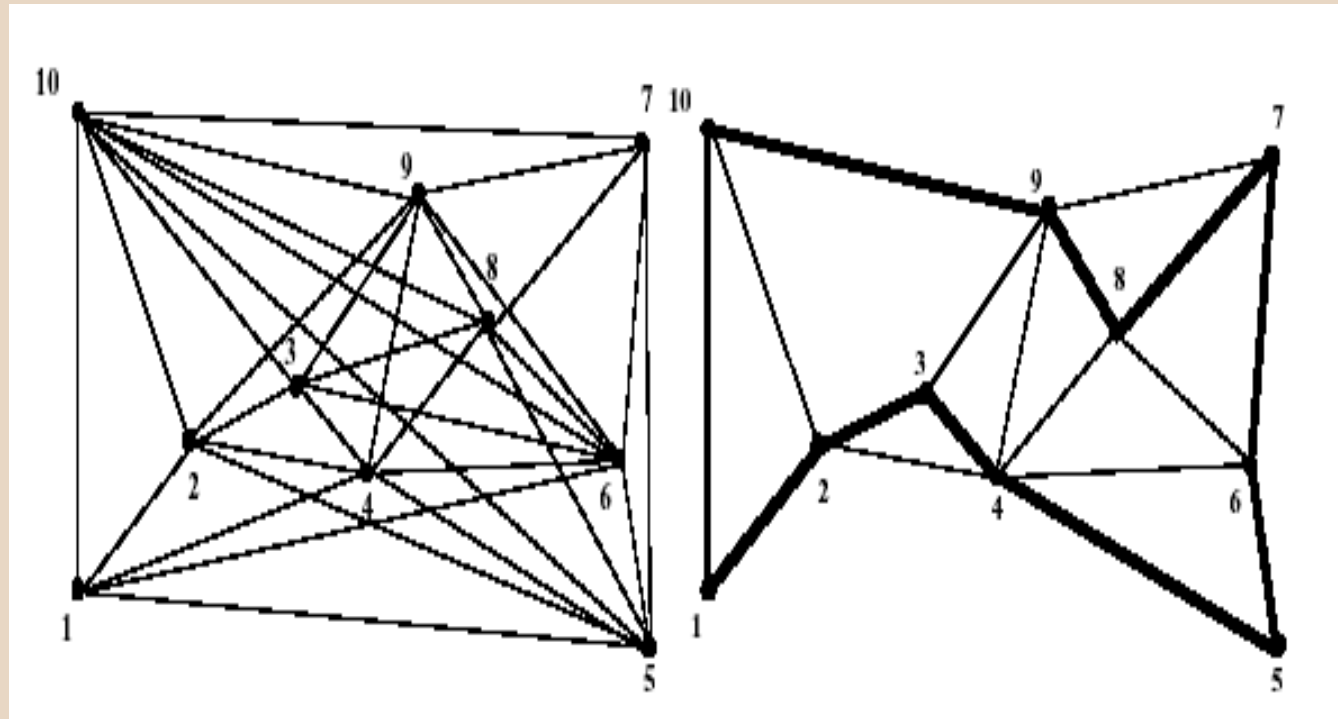
# ALGORITMOS DE HORMIGAS Y EL TSP

- Después de cada recorrido de una hormiga, se actualiza cada arco:

$$T_{ij}(t + n) = p \cdot T_{ij}(t) + \Delta T_{ij}$$

# CRITERIO DE DETENCIÓN

- Estancamiento (stagnation).
- Máximo de Iteraciones.



# UN PROGRAMA ACO PUEDE LLEGAR A SER BASTANTE COMPLEJO

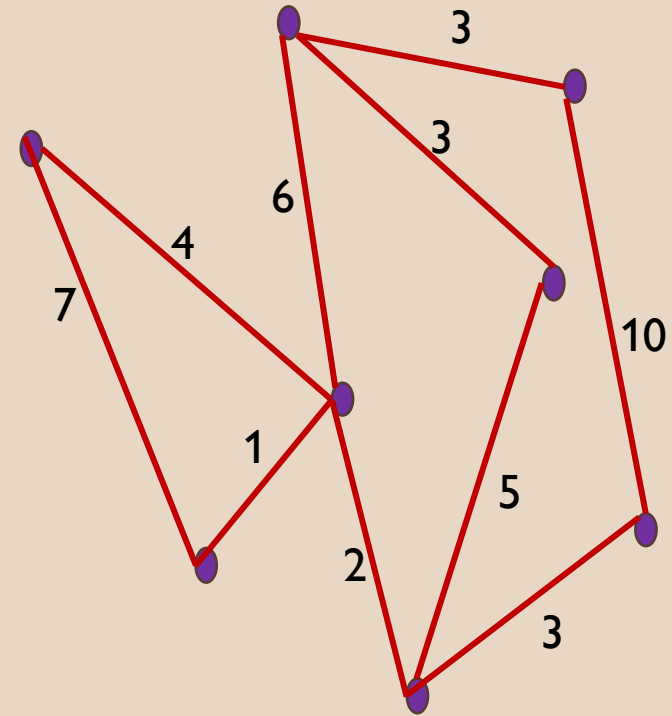
- Por la cantidad de funciones involucradas.
- La buena noticia, es que se puede modularizar, y trabajar en pequeñas funciones

# INICIALIZACIÓN DE FEROMONA

```
• import random
•
  # Definir la longitud del camino
• longitud_arreglo = 10
•
  # Crear un arreglo vacío
• arreglo = []
•
  # Llenar el valor de los arcos con valores de feromona enteros aleatorios entre 1 y 10
• for _ in range(longitud_arreglo):
•   valor_aleatorio = random.randint(1, 10) # Generar un valor aleatorio entre 1 y 10 (ambos inclusive)
•   arreglo.append(valor_aleatorio) # Agregar el valor al arreglo
•
  # Imprimir el arreglo resultante
• print("Camino con valores de feromona asociados a los arcos para 10 aristas:", arreglo)
```

# EL RESULTADO

- [6, 7, 3, 2, 1, 4, 10, 3, 5, 3]



# TODAS LAS FUNCIONES RESULTAN SENCILLAS, SI SE ATOMIZAN

- El próximo ejemplo muestra cómo hacer una selección de un posible destino de una hormiga, en base a la evaluación que hace de los caminos que llevan a distintos puntos.

# CÓDIGO

```
# En este caso voy a definir arbitrariamente los puntos y sus valoraciones
```

```
puntos = { 'A': 8, 'B': 5, 'C': 9, 'D': 3, 'E': 7, 'F': 6 }
```

```
# Imprimir los puntos y sus valoraciones
```

```
print("Puntos y sus valoraciones:")
```

```
for punto, valoracion in puntos.items():
```

```
    print(f"{punto}: {valoracion}")
```

```
# Seleccionar el punto con la valoración más alta
```

```
punto_seleccionado = max(puntos, key=puntos.get)
```

```
# Imprimir el punto seleccionado
```

```
print(f"\nEl punto seleccionado es: {punto_seleccionado} con valoración  
{puntos[punto_seleccionado]}")
```

# Y EL RESULTADO ES

Puntos y sus valoraciones:

A: 8

B: 5

C: 9

D: 3

E: 7

F: 6

El punto seleccionado es: C con valoración 9

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A |   | 2 | 7 | 6 | 1 | 3 |
| B | 2 |   | 3 | 4 | 6 | 5 |
| C | 4 | 2 |   | 5 | 4 | 6 |
| D | 6 | 7 | 7 |   | 4 | 1 |
| E | 8 | 6 | 5 | 3 |   | 2 |
| F | 7 | 1 | 3 | 6 | 5 |   |

La matriz de feromonas es una parte de la evaluación

# ESTRUCTURA BÁSICA

Crear un algoritmo de colonia de hormigas puede ser un proyecto complejo, pero podemos empezar con la estructura básica del código. A continuación, un esbozo simple en Python que se puede usar como punto de partida.

Este esquema se centra en la estructura del código y deja muchos detalles específicos a ser llenados de acuerdo a las necesidades del problema.

Este esquema proporciona las clases básicas ([Grafo](#), [Hormiga](#), y [AlgoritmoHormigas](#)) que se pueden expandir y modificar según las necesidades específicas. La implementación del método [seleccionar\\_siguiente\\_nodo](#) y la lógica para actualizar las feromonas son áreas críticas que deberán personalizarse para adaptarse a la lógica específica del problema particular.

```
import random
```

```
class Grafo:
```

```
    def __init__(self, nodos, aristas):
```

```
        self.nodos = nodos
```

```
        self.aristas = aristas
```

```
    # Se puede agregar más métodos según sea necesario para representar y manipular el grafo
```

```
class Hormiga:
```

```
    def __init__(self, grafo, nodo_inicial):
```

```
        self.grafo = grafo
```

```
        self.nodo_actual = nodo_inicial
```

```
        self.visitados = set()
```

```
def seleccionar_siguiente_nodo(self):
```

```
# Implementa la lógica para seleccionar el próximo nodo basado en feromonas y heurísticas
```

```
pass
```

```
# Se puede agregar más métodos según sea necesario para que la hormiga realice acciones específicas
```

```
class AlgoritmoHormigas:
```

```
    def __init__(self, grafo, num_hormigas, num_iteraciones):
```

```
        self.grafo = grafo
```

```
        self.num_hormigas = num_hormigas
```

```
        self.num_iteraciones = num_iteraciones
```

```
        self.feromonas = {}
```

```
# Se puede inicializar las feromonas según las necesidades del problema
```

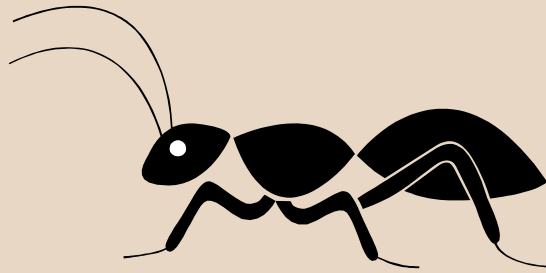
```
def ejecutar(self):  
    for _ in range(self.num_iteraciones):  
        hormigas = [Hormiga(self.grafo, nodo_inicial=random.choice(self.grafo.nodos))  
                     for _ in range(self.num_hormigas)]  
  
        for _ in range(len(self.grafo.nodos) - 1):  
            for hormiga in hormigas:  
                siguiente_nodo = hormiga.seleccionar_siguiente_nodo()  
  
                if siguiente_nodo is not None:  
                    hormiga.visitados.add(hormiga.nodo_actual)  
                    hormiga.nodo_actual = siguiente_nodo  
            for hormiga in hormigas:  
                hormiga.visitados.add(hormiga.nodo_actual)  
        # Implementa la lógica para actualizar feromonas basado en el camino de cada hormiga
```

```
if __name__ == "__main__":  
    # Define tu grafo con nodos y aristas  
    grafo = Grafo(nodos=[0, 1, 2, 3], aristas=[(0, 1), (1, 2), (2, 3), (3, 0)])  
  
    # Configura y ejecuta el algoritmo de hormigas  
    algoritmo = AlgoritmoHormigas(grafo, num_hormigas=5, num_iteraciones=100)  
    algoritmo.ejecutar()
```

# ALGORITMOS DE HORMIGAS Y EL TSP

- Travelling Salesman Problem (TSP)
- Ruteamiento vehicular
- Stock Cutting

# VENTAJAS Y DESVENTAJAS



# VENTAJAS Y DESVENTAJAS

- Para los problemas TSPs (Traveling Salesman Problem), relativamente eficiente
  - Para un número reducido de nodos, TSPs puede resolverse por búsqueda exhaustiva
  - Para un número grande de nodos, TSPs resultan computacionalmente difíciles de resolver (NP-hard)
    - tiempo exponencial de convergencia
- Se ha obtenido buenos resultados en comparación con otras técnicas de optimización sobre TSPs (redes neuronales, algoritmos genéticos, simulated annealing)
- En comparación con AGs:
  - Retiene en memoria la colonia entera (no solamente la última generación)
  - Es poco afectada por la existencia de soluciones iniciales pobres (debido a la combinación de selección aleatoria de trayectorias y memoria de la colonia)

# VENTAJAS Y DESVENTAJAS (CONT)

- Se puede usar en aplicaciones dinámicas (se adapta a cambios tales como nuevas distancias, etc.).
- Tiene una gran variedad de aplicaciones.
- Tal como con AGs, son una buena opción para problemas discretos con restricciones (no para algoritmos basados en gradiente).

# VENTAJAS Y DESVENTAJAS (CONT)

- Análisis teórico es difícil:
  - Debido a la secuencia de decisiones random (no independientes).
  - Probabilidad de distribución cambia con cada iteración.
  - La investigación es experimental, no teórica.
- La convergencia está garantizada, pero el tiempo necesario es algo incierto.

# VENTAJAS Y DESVENTAJAS (CONT.)

- Compromisos en la evaluación de convergencia:
  - En problemas NP-hard, se requieren soluciones de alta calidad rápidamente – el foco está en la calidad de las soluciones.
  - En problemas con redes dinámicas de ruteamiento, se necesitan soluciones en condiciones cambiantes – el foco está en la evaluación efectiva de rutas alternativas.

# VENTAJAS Y DESVENTAJAS (CONT.)

- La codificación puede ser compleja, no necesariamente intuitiva
  - Incrementos y decrementos de los rastros de feromona incluyen actualizaciones globales y locales.
  - Existen muchas versiones de algoritmos ACO que apuntan a diferentes especificidades en los problemas.

# ¡OH NO! MÁS HORMIGAS..

- Un ejemplo adicional

# REPITAMOS LAS IDEAS BÁSICAS

Las hormigas son agentes que :

Se mueven entre nodos en un grafo.

Escogen dónde ir basándose en la *intensidad* de la fermona (y en otras cosas).

El recorrido de una hormiga representa una solución candidata específica.

Cuando una hormiga completa una solución candidata, la feromona permanece en el camino, de acuerdo a la calidad de la solución.

Esto afecta el comportamiento de otras hormigas por `stigmergy' ...

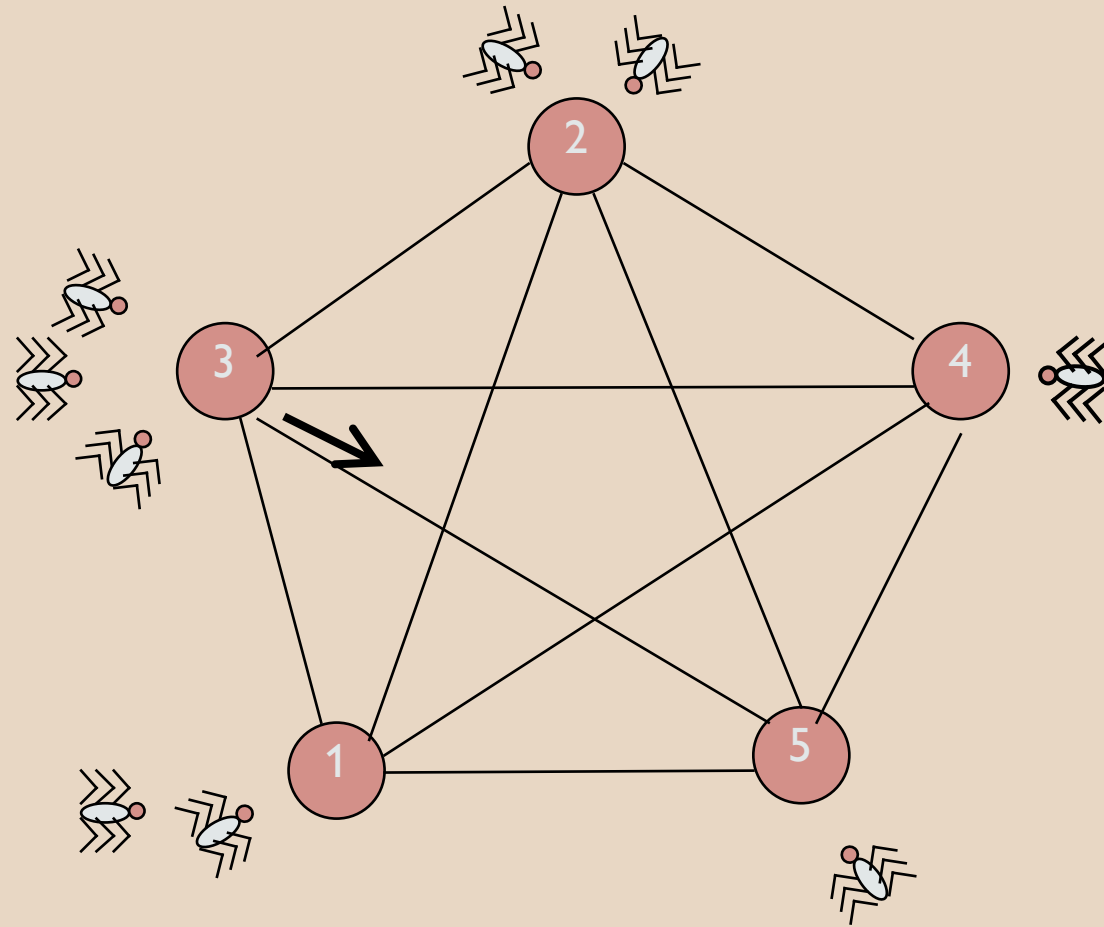
# CONSIDERACIONES PARA SU APLICACIÓN

- El enfoque ACO es particularmente adecuado para ser aplicado a problemas que acepten una **representación vía grafo** (necesario para imitar la búsqueda de un camino).
- **Representación del rastro** de feromona y su asociación a las conexiones entre las componentes del problema.
- Posibilidad de añadir **conocimiento del problema (heurística local)** para guiar junto con el rastro la construcción de las soluciones.

# CONSIDERACIONES PARA SU APLICACIÓN

- Un ejemplo clásico es el problema TSP
- **Representación del rastro** de feromona puede ser realizado a través de una matriz de números reales ( $\tau$ ) de  $n \times n$ .
- **Heurística local:**  $1/d_{ij}$ , es decir, un valor inversamente proporcional a la distancia entre las ciudades  $i$  y  $j$ .

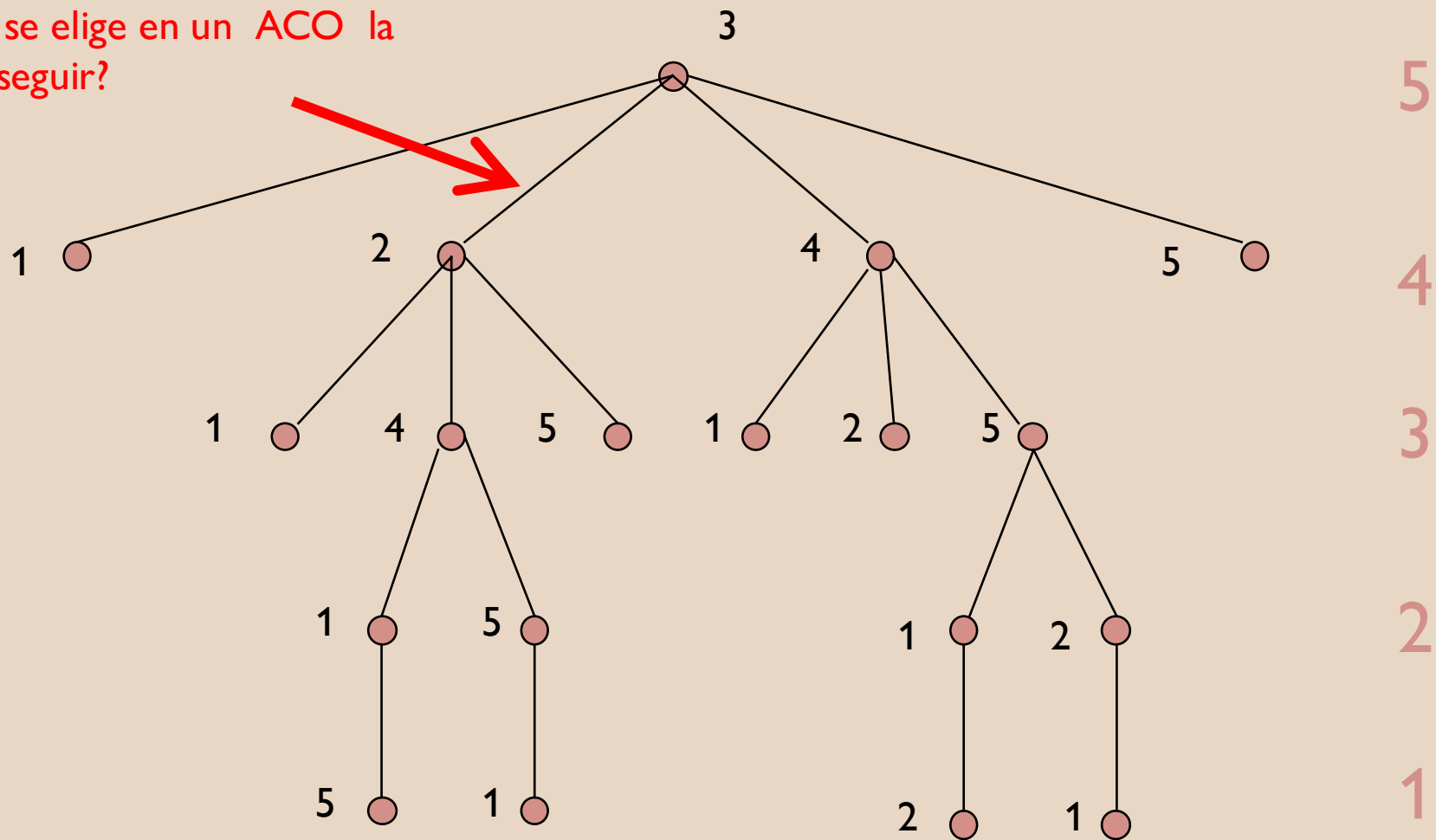
# ACO APLICADO A TSP



# Espacio de Búsqueda - TSP

## Algoritmo ACO

¿Cómo se elige en un ACO la rama a seguir?



# EL PRIMER ALGORITMO ACO (ANT SYSTEM OAS)

```
Inicializar();  
for c=1 to Nro_ciclos  
{  
    for k=1 to Nro_ants  
        ant-k construye solución k;  
        Guardar la mejor solución;  
        Actualizar Rastro (i.e.,  $\tau_{ij}$ );  
        Reubicar hormigas para el próximo ciclo;  
    }  
Imprimir la mejor solución encontrada;
```

# EL PRIMER ALGORITMO ACO (ANT SYSTEM O AS)

```
Inicializar();  
for c=1 to Nro_ciclos  
{  
    for k=1 to Nro_ants  
        ant-k construye solución k;  
        Guardar la mejor solución;  
        Actualizar Rastro (i.e.,  $\tau_{ij}$ );  
        Reubicar hormigas para el próximo ciclo;  
    }  
Imprimir la mejor solución encontrada;
```

La construcción se realiza  
paso a paso en forma  
probabilística considerando

$\tau_{ij}$  y  $\eta_{ij}$

# AS - CONSTRUCCIÓN DE UNA SOLUCIÓN PARA TSP

/\*  $S_k$ : Solución o permutación construida por la hormiga  $k$  \*/

$S_k = \text{Ciudad\_Inicial}$ ; (escogida de acuerdo a algún criterio)

mientras no se haya completado el tour

{

    Seleccionar próx. Ciudad ( $j$ ) con probabilidad:

    ( $i$  es la última ciudad incluida)

$$P_{ij}(k) = \begin{cases} \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{h \in \text{NoVisitadas}} \tau_{ih}^{\alpha} \cdot \eta_{ih}^{\beta}} & j \in \text{NoVisitadas} \\ 0 & \text{en otro caso} \end{cases}$$

$S_k = S_k \oplus j$

}

# EL PRIMER ALGORITMO ACO (ANT SYSTEM O AS)

```
Inicializar();  
for c=1 to Nro_ciclos  
{  
    for k=1 to Nro_ants  
        ant-k construye solución k;  
        Guardar la mejor solución;  
        Actualizar Rastro (i.e.,  $\tau_{ij}$ );  
        Reubicar hormigas para el próximo ciclo;  
    }  
    Imprimir la mejor solución encontrada;
```

Se puede hacer considerando todas las soluciones encontradas o un subconjunto de ellas

# ACTUALIZACIÓN DEL RASTRO EN AS

Acumulación de rastro proporcional a la calidad de las soluciones (i.e.,  $NroAnts$  soluciones):

$$\Delta \tau_{ij}(t+1) = \sum_{k=1}^{NroAnts} \Delta^k \tau_{ij}$$

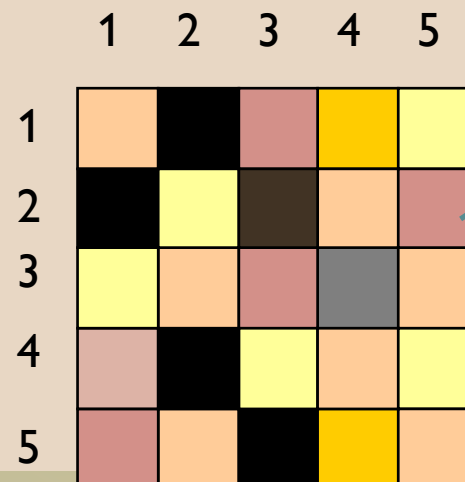
Este valor es calculado directamente proporcional a la calidad de la solución

Actualización Efectiva ( $\rho$  es el factor de persistencia del rastro)

$$\tau_{ij}(t+1) = \rho \cdot \tau_{ij}(t) + \Delta \tau_{ij}(t+1)$$

# IMPORTANCIA DE RASTRO ( $\tau$ )

- Como todo método heurístico, un algoritmo ACO tiene su **bloque de construcción** a partir del cual se generan nuevas soluciones del espacio de búsqueda.
- El bloque de construcción está representado por la estructura  $\tau$  dado que incide directamente en las componentes a seleccionar.



El nivel de feromona indica la fortaleza de la conexión.

# UN EJEMPLO SENCILLO

```
import random
```

```
# Parámetros del algoritmo
```

```
num_hormigas = 10
```

```
num_iteraciones = 100
```

```
num_ciudades = 20
```

```
feromonas = [[1.0] * num_ciudades for _ in range(num_ciudades)]
```

```
alfa = 1.0 # Peso de las feromonas
```

```
beta = 2.0 # Peso de la visibilidad
```

```
evaporacion = 0.5 # Tasa de evaporación de feromonas
```

```
# Datos del problema (pueden variar según el problema específico)
```

```
num_ciudades = 20
```

```
distancias = [[random.randint(1, 100) for _ in range(num_ciudades)] for _ in range(num_ciudades)]
```

# Función de inicialización de feromonas

```
def inicializar_feromonas(num_ciudades):
```

```
    return [[1.0] * num_ciudades for _ in range(num_ciudades)]
```

# Función de actualización de feromonas

```
def actualizar_feromonas(feromonas, rutas, costos):
```

```
    for i in range(len(feromonas)):
```

```
        for j in range(len(feromonas[i])):
```

```
            feromonas[i][j] *= (1.0 - evaporacion)
```

```
            for k in range(num_hormigas):
```

```
                feromonas[i][j] += 1.0 / costos[k]
```

```
    return feromonas
```

```
# Función para seleccionar la siguiente ciudad basada en probabilidad
```

```
def seleccionar_ciudad(actual, feromonas, distancias, alfa, beta):
```

```
    opciones = [i for i in range(len(feromonas[actual]))]
```

```
    opciones.remove(actual)
```

```
    probabilidad = [((feromonas[actual][i] ** alfa) * ((1.0 / distancias[actual][i]) ** beta)) for i in opciones]
```

```
    suma_probabilidad = sum(probabilidad)
```

```
    probabilidad = [p / suma_probabilidad for p in probabilidad]
```

```
    seleccion = random.choices(opciones, weights=probabilidad)[0]
```

```
    return seleccion
```

```
# Algoritmo principal de la colonia de hormigas
```

```
def colonia_hormigas(num_hormigas, num_iteraciones, feromonas, alfa, beta):
```

```
    for _ in range(num_iteraciones):
```

```
        rutas = []
```

```
        costos = []
```

```
        for _ in range(num_hormigas):
```

```
            # Inicializar una hormiga en una ciudad aleatoria
```

```
            hormiga = random.randint(0, num_ciudades - 1)
```

```
            ruta = [hormiga]
```

```
            costo_total = 0
```

```
            # Construir la ruta de la hormiga
```

```
            for _ in range(num_ciudades - 1):
```

```
                siguiente_ciudad = seleccionar_ciudad(hormiga, feromonas, distancias, alfa, beta)
```

```
                ruta.append(siguiente_ciudad)
```

```
                costo_total += distancias[hormiga][siguiente_ciudad]
```

```
                hormiga = siguiente_ciudad
```

```
# Agregar la última conexión a la primera ciudad
```

```
ruta.append(ruta[0])
```

```
costo_total += distancias[hormiga][ruta[0]]
```

```
# Almacenar la ruta y el costo total de la hormiga
```

```
rutas.append(ruta)
```

```
costos.append(costo_total)
```

```
# Actualizar feromonas
```

```
feromonas = actualizar_feromonas(feromonas, rutas, costos)
```

```
# Encontrar la mejor ruta y su costo mínimo
```

```
mejor_ruta = rutas[costos.index(min(costos))]
```

```
mejor_costo = min(costos)
```

```
print(f"Iteración {_ + 1}: Mejor Ruta = {mejor_ruta}, Costo = {mejor_costo}")
```

```
return mejor_ruta, mejor_costo
```

```
# Inicializar feromonas
```

```
feromonas = inicializar_feromonas(num_ciudades)
```

```
# Ejecutar el algoritmo de colonia de hormigas
```

```
mejor_ruta, mejor_costo = colonia_hormigas(num_hormigas, num_iteraciones, feromonas, alfa, beta)
```

```
print(f"\nMejor Ruta Final: {mejor_ruta}")
```

```
print(f"Costo de la Mejor Ruta: {mejor_costo}")
```

# RESULTADOS DE LA EJECUCIÓN

....

Iteración 19: Mejor Ruta = [16, 7, 1, 12, 9, 2, 11, 15, 19, 14, 10, 13, 17, 15, 0, 16, 7, 1, 12, 10, 16],  
Costo = 148

....

Mejor Ruta Final: [18, 7, 1, 12, 10, 13, 14, 10, 13, 17, 15, 18, 8, 13, 17, 15, 0, 16, 7, 1, 18]

Costo de la Mejor Ruta: 135

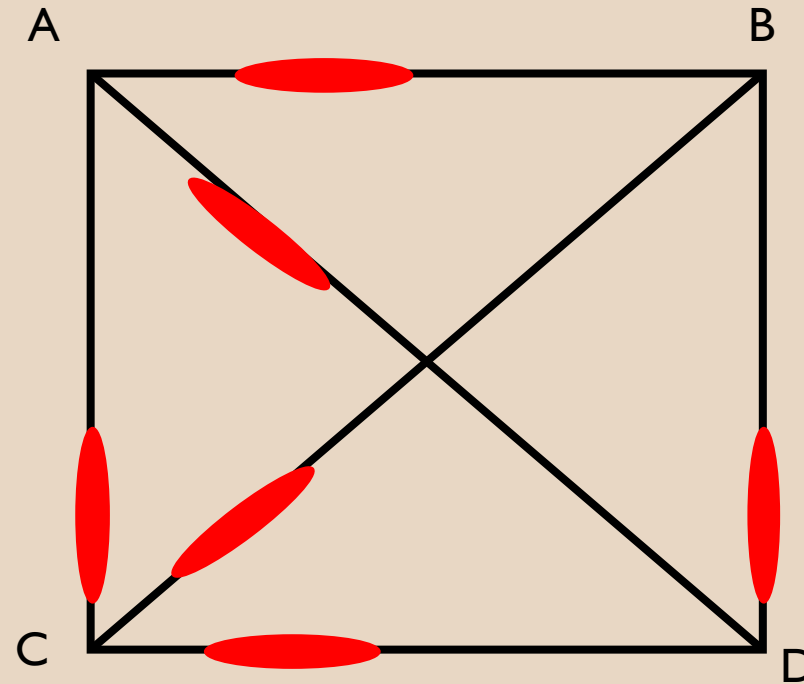
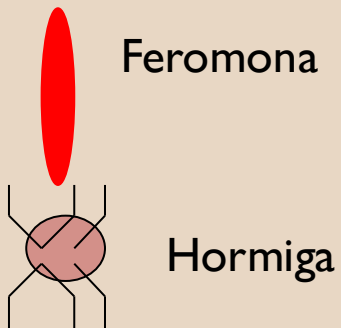
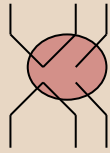
# OTROS ALGORITMOS ACO

Surgen como respuesta a ciertos problemas observados en AS y básicamente se diferencian en cómo usan y/o modifican el rastro de feromona.

- MinMax-AS (control sobre los valores del rastro)
- AS-rank (ranking de soluciones)
- AS-elistim (solo la mejor solución)
- Ant Colony System (ACS)
- Ant-Q (basado en Q-Learning)

# TSP CON 4 CIUDADES

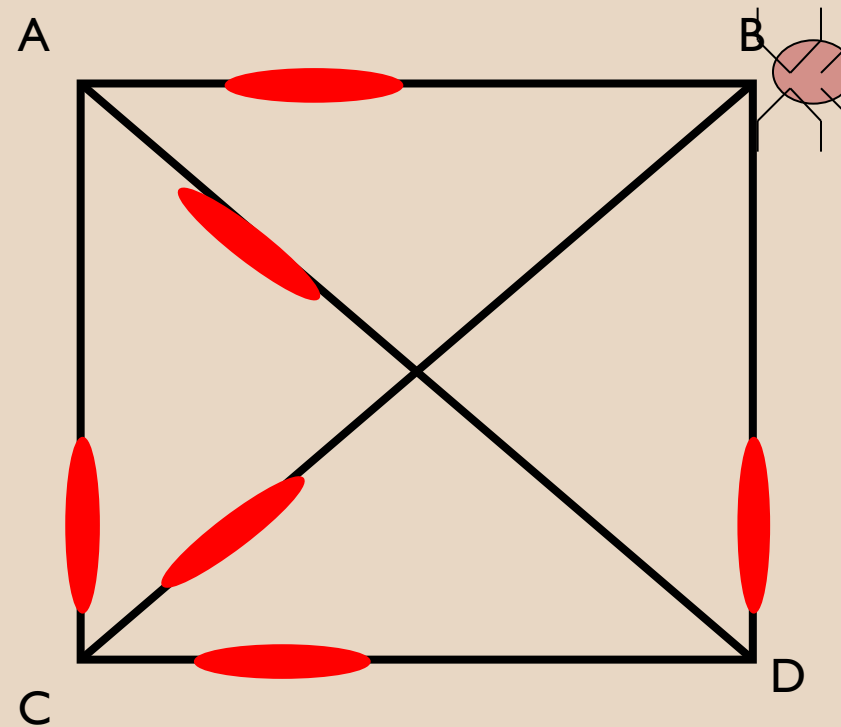
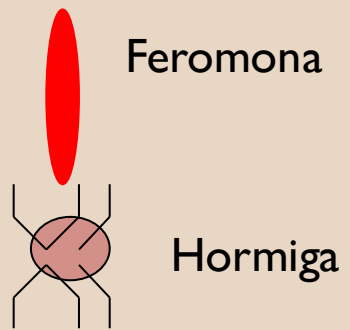
Inicialmente (ojo esta es una variación) se pueden depositar randómicamente niveles arbitrarios de feromona en los arcos



AB: 10, AC: 10, AD: 30, BC: 40, CD: 20

# TSP CON 4 CIUDADES

Se elige (random) un nodo para ubicar una hormiga



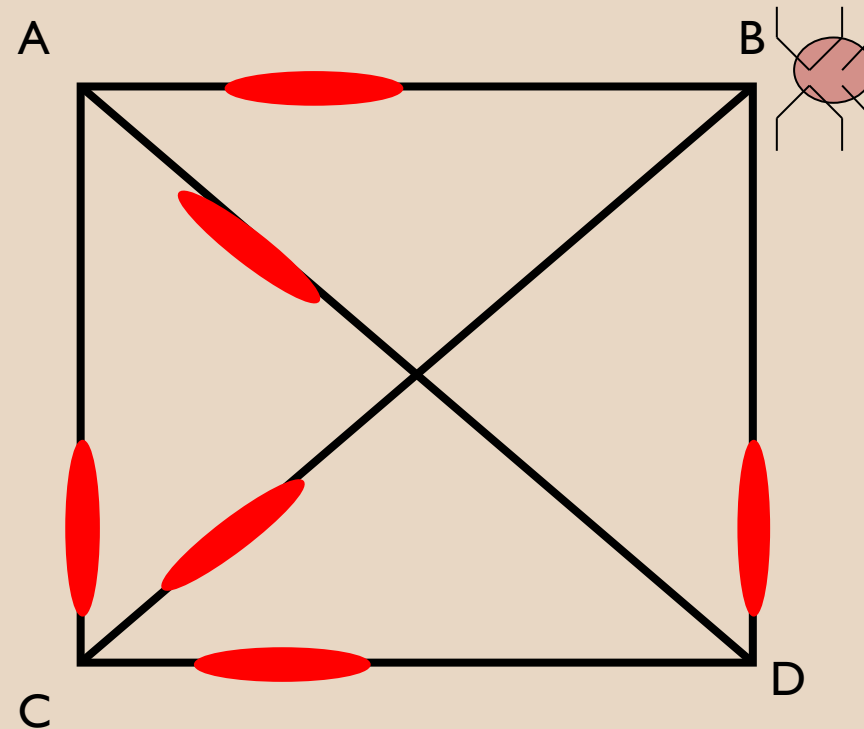
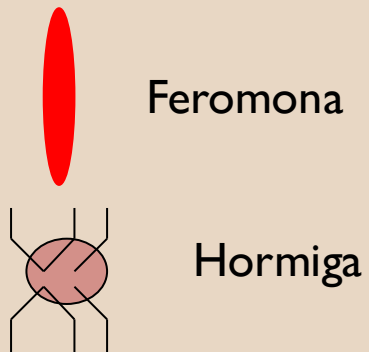
AB: 10, AC: 10, AD: 30, BC: 40, CD: 20

# TSP CON 4 CIUDADES

La hormiga “decide” dónde ir en base a probabilidades que se calculan en base a:

- Intensidad de la feromona
- Distancia al próximo nodo

Supongamos que se elige el arco BC



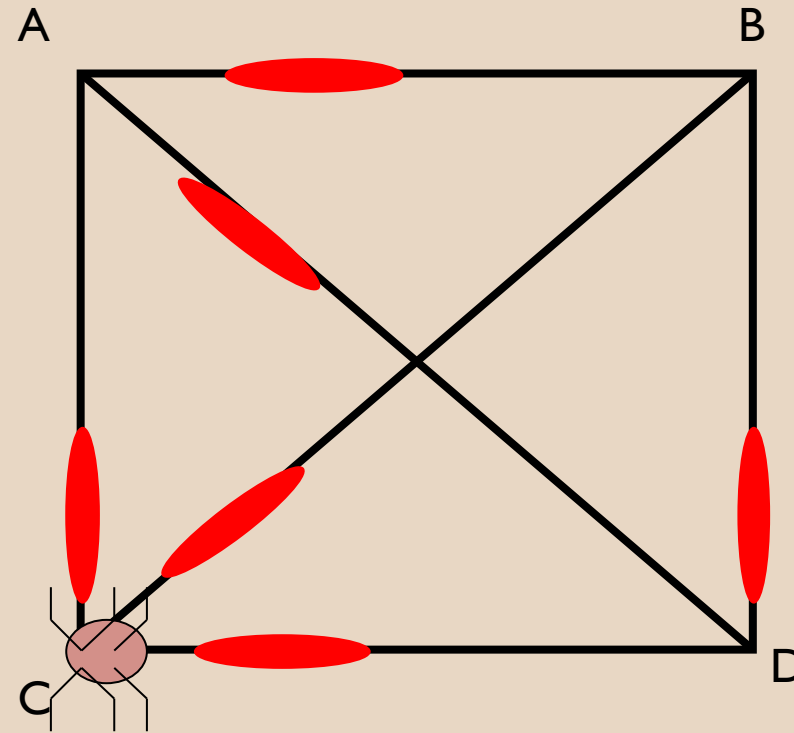
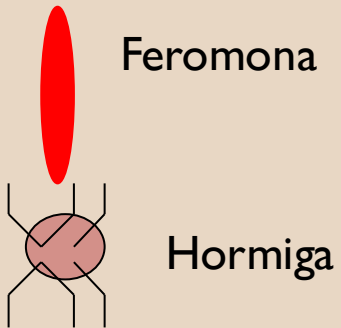
AB: 10, AC: 10, AD: 30, BC: 40, CD: 20

# TSP CON 4 CIUDADES

La hormiga ahora está en C y tiene memoria  
(`tour memory' = {B, C}) – por lo que no puede  
visitar ni B ni C.

Elige el próximo arco (de entre aquellos permitidos)  
En base a la intensidad de la feromona y la distancia.

Supongamos que elige CD

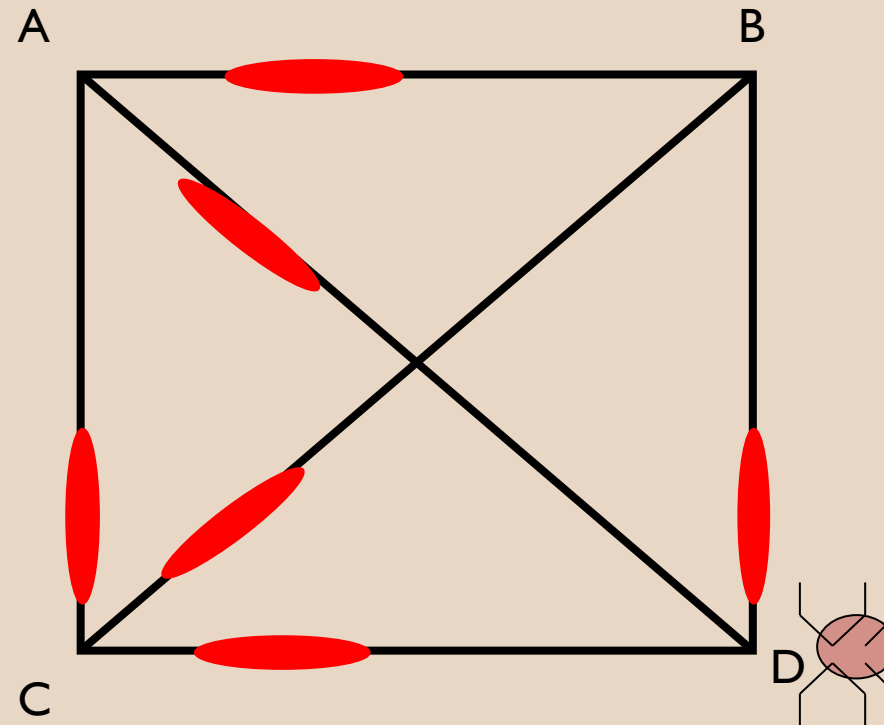


AB: 10, AC: 10, AD: 30, BC: 40, CD: 20

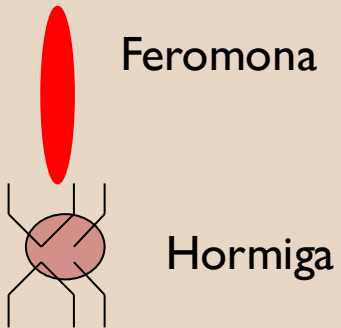
# TSP CON 4 CIUDADES

La hormiga está ahora en D, y en su memoria tiene como  
'tour memory' = {B, C, D}

Ahora hay un solo lugar al que puede ir



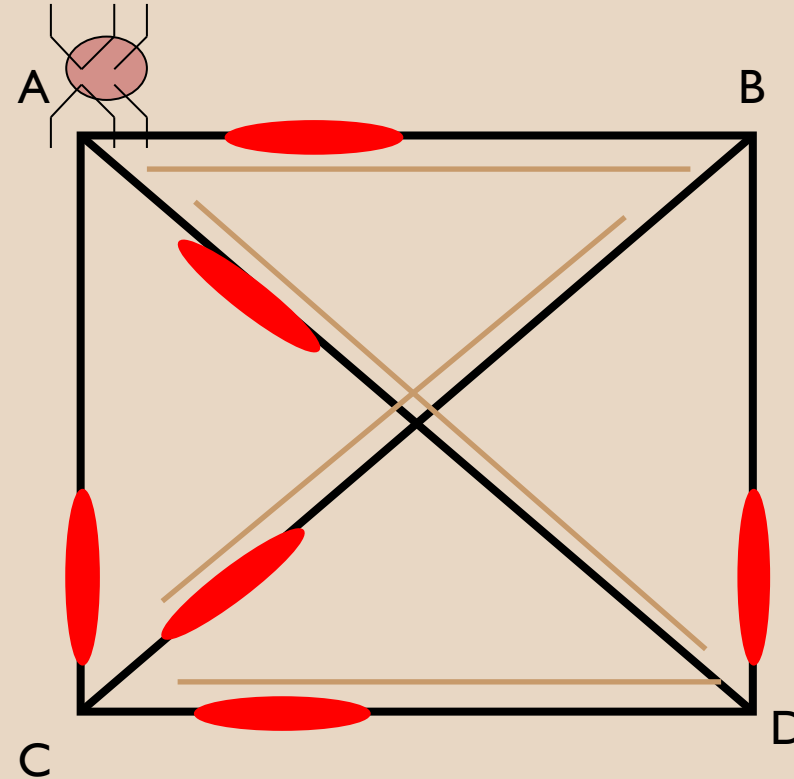
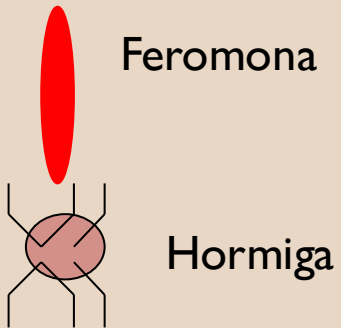
AB: 10, AC: 10, AD: 30, BC: 40, CD: 20



# TSP CON 4 CIUDADES

Así, la hormiga finaliza su recorrido que considera  
Los arcos BC, CD, y DA.  
Se agrega el arco AB para completar el tour.

Y ahora, se incrementa la  
feromona en los arcos recorridos,  
de acuerdo al fitness del tour

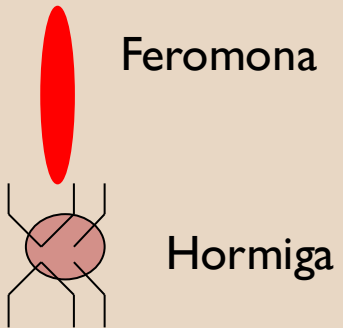
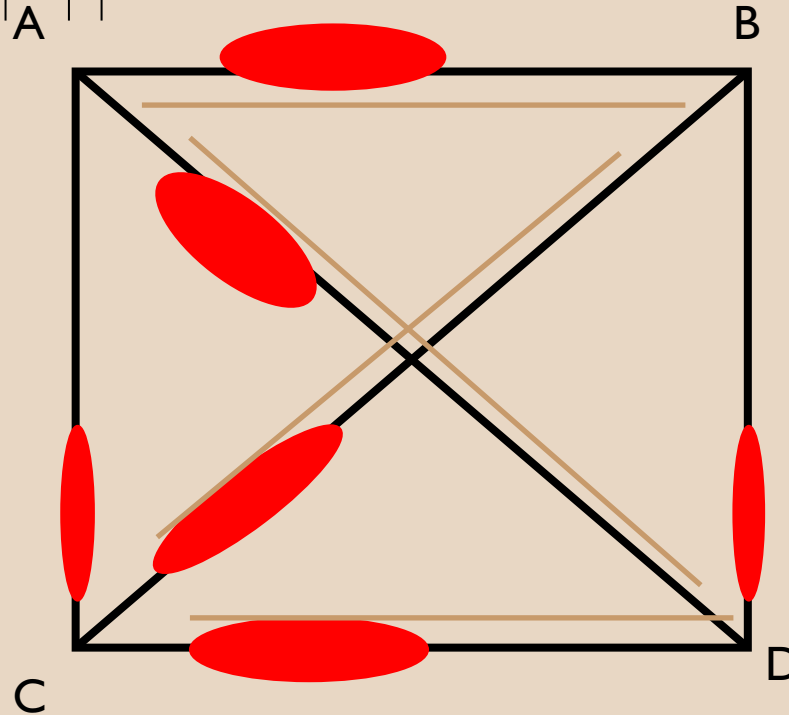


AB: 20, AC: 10, AD: 40, BC: 50, CD: 30

# TSP CON 4 CIUDADES



Luego, la feromona es disminuida un poco en todos los arcos, para modelar el decaimiento de la intensidad de la feromona con el paso del tiempo

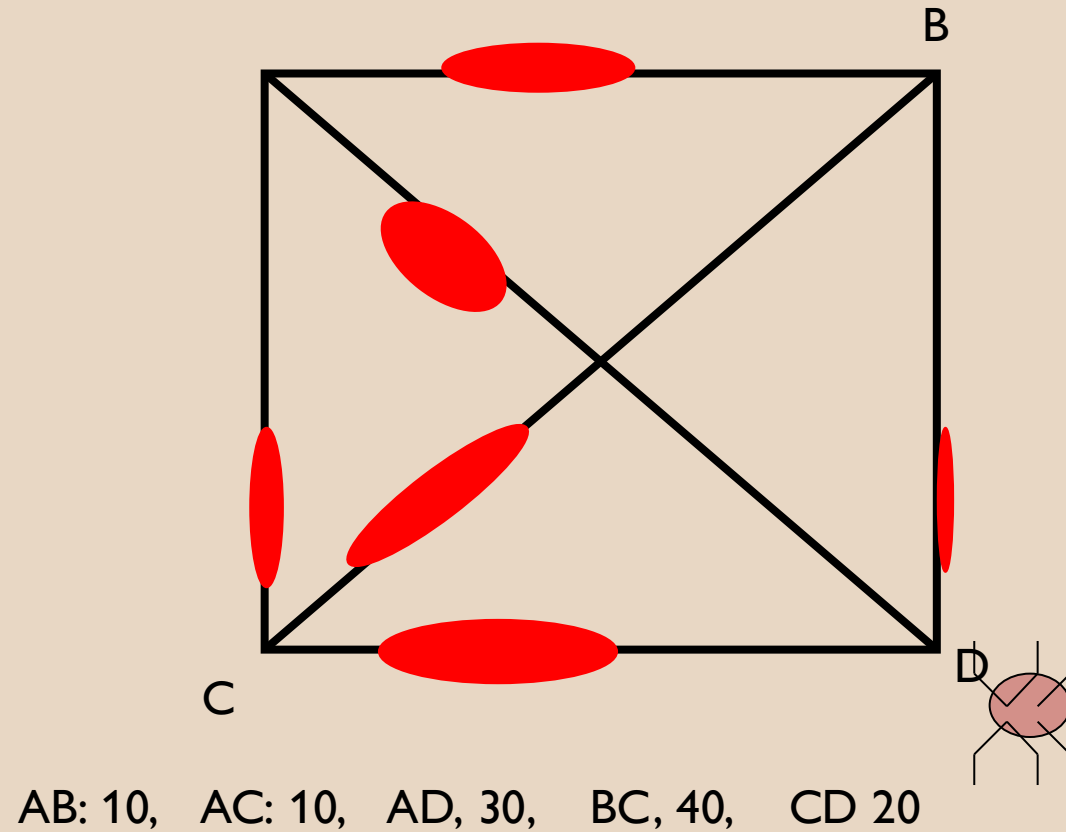
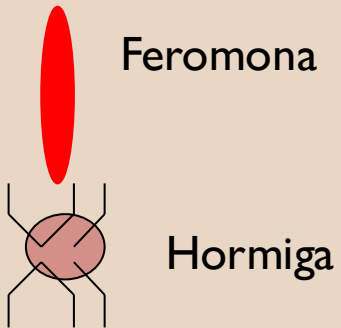


AB: 15, AC: 5, AD: 35, BC: 45, CD: 25

# TSP CON 4 CIUDADES

Partimos de nuevo, con otra hormiga en una posición random.

¿Hacia dónde vamos?



# EL ALGORITMO ACO PARA EL TSP

(VERSION SIMPLIFICADA CON DETALLES ESENCIALES)

Tenemos  $n$  ciudades.

1. Colocamos algunas hormigas en cada ciudad. Cada hormiga:

- Completa un ciclo con las ciudades, volviendo al origen, y usa una regla de transición para decidir por qué arco continuar. Con esta regla la elección es random, pero “influida” parcialmente por los niveles de feromona en cada arco, y parcialmente por “*información heurística*.”

2. Cuando todas las hormigas han completado sus tours. Se actualiza globalmente la feromona.

- Se reducen los niveles actuales en todos los arcos (los niveles de feromona decaen en el tiempo).
- La feromona se distribuye (con retraso) para cada hormiga de la siguiente manera: se coloca feromona en aquellos links que forman su tour con una intensidad que depende de la bondad del tour.

Luego, se vuelve a 1 y se repite el proceso completo (muchas veces), hasta alcanzar la condición de término.

# LA REGLA DE TRANSICIÓN

$T(r,s)$  es la cantidad de feromona que hay en el arco que va directamente desde la ciudad  $r$  a la ciudad  $s$

$H(r,s)$  es el valor *heurístico* del arco – en el TSP clásico, se elige como  $1/\text{distancia}(r,s)$  -- i.e. mientras más corta es la distancia, mayor es su valor

$p_k(r,s)$  es la probabilidad de que una hormiga  $k$  escoja un arco que va desde  $r$  a  $s$

$\beta$  es un *parámetro* que podemos llamar *intensidad de la heurística*

La regla es:

$$p_k(r,s) = \frac{T(r,s) \cdot H(r,s)^\beta}{\sum T(r,c) \cdot H(r,c)^\beta}$$

Ciudades no visitadas

Donde la hormiga está en la ciudad  $r$ , y  $s$  es una ciudad aún no visitada. La sumatoria es sobre las ciudades no visitadas

# ACTUALIZACION GLOBAL DE LA FEROMONA

$A_k(r,s)$  es la cantidad de feromona agregada a  $(r, s)$  por la hormiga  $k$ .

$m$  es el número de hormigas

$\rho$  es un parámetro llamado tasa de evaporación de la feromona

y  $L_k$  es la longitud del tour completado por la hormiga  $k$

$T(r, s)$  en la próxima generación es:

$$\rho \cdot T(r, s) + \sum_{k=1}^m A_k(r, s)$$

Donde  $A_k(r, s) = 1 / L_k$

¿Cómo creen ustedes que los parámetros  $m$ ,  $\beta$ ,  $\rho$  etc ... afectan la búsqueda?

# Optimización por Colonia de Hormigas (ACO)

Proceso de feromonas para encontrar el camino más corto

## 1. Exploración



Las hormigas salen aleatoriamente desde el nido buscando comida, explorando múltiples caminos sin dirección definida.

## 2. Dejar feromonas



Al encontrar la comida, la hormiga regresa dejando feromonas en su camino. Cada hormiga libera feromonas como rastro químico.

## 3. Reforzar camino



El camino más corto se recorre en menos tiempo, las feromonas se acumulan y se refuerzan con cada pasada.

## 4. Convergencia a la ruta óptima



Con el tiempo, la mayoría de hormigas siguen el camino más corto. Las otras rutas se abandonan al evaporarse las feromonas.



Hormiga

..... Feromona (rastros químicos)

↗ Dirección del movimiento

Clave: Las hormigas depositan feromonas en el recorrido → Otras hormigas siguen los caminos con mayor concentración de feromonas → El camino más corto se refuerza con el tiempo

# ALGORITMOS DE HORMIGAS - APLICACIONES

- Marco Dorigo, mantiene una página web dedicada a este tema:

<http://iridia.ulb.ac.be/~mdorigo/ACO/ACO.html>

- Contiene información sobre algoritmos y links a los artículos relevantes publicados sobre este tema.

# RESUMEN DEL ALGORITMO (1)

- Inicialización
- Ciclo iterativo (hasta cumplir condición de término)
  - Construir soluciones.
  - Actualizar feromonas.
  - Guardar la mejor solución.
- Retornar la mejor ruta

# RESUMEN DEL ALGORITMO (2).

## INICIALIZACIÓN

- Crear grafo
  - Nodos son ciudades o puntos de interés.
  - Aristas son caminos con costos o distancias.
  - Inicializar feromonas.
    - $\tau_{ij}$ . Se coloca una cantidad pequeña, igual en todas las aristas
- Parámetros
  - $\alpha$ : Peso de la feromona
  - $\beta$  Peso de la distancia
  - $\rho$  Tasa de evaporación
  - m número de hormigas

# RESUMEN DEL ALGORITMO (3). CICLO ITERATIVO.

- Paso 1
  - Construir soluciones.
- Paso 2
  - Actualizar feromonas.

# RESUMEN DEL ALGORITMO (4). CONSTRUIR SOLUCIONES.

- Para cada hormiga  $k$ 
  - Modo random
    - Elegir un nodo no seleccionado aún, en base a una probabilidad.
    - Guardar ruta y costo de la hormiga.

# RESUMEN DEL ALGORITMO (5). ACTUALIZAR FEROMONAS.

- Para todas las aristas
  - Evaporar una cierta cantidad de feromona
    - Por ejemplo, un cierto porcentaje.
  - Depositar feromona
    - Considerando feromona actual y longitud de la ruta.

Las mejores rutas tendrán más feromona.

# RESUMEN DEL ALGORITMO (6). CASI FINAL

- Guardar la major ruta encontrada.
- Retornar la major ruta

# RESUMEN DEL ALGORITMO (7). FINAL

- Inicio
- Inicializar feromonas y parámetros.
- Repetir hasta n iteraciones.
  - Cada hormiga construye una ruta.
  - Evaluar costo de las rutas.
  - Actualizar feromonas y evaporar.
- Retornar mejor solución.

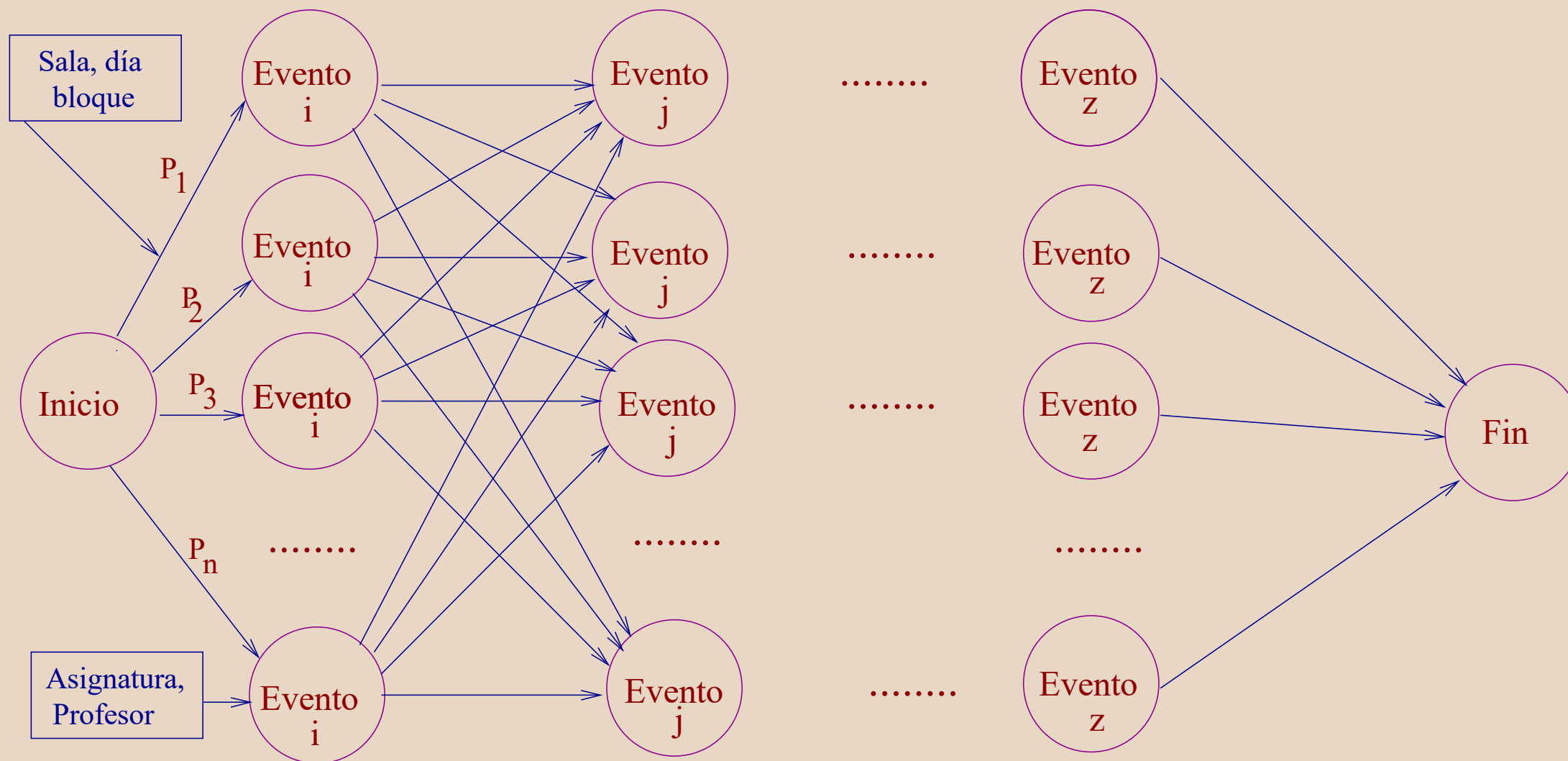
# APLICACIONES REALES (AJENAS Y PROPIAS)

- ABC (Ant Based Control).
  - Se utilizó para ruteamiento de llamadas en una red telefónica.
  - Es el primer algoritmo de hormigas usado para redes de telecomunicaciones
    - 1996
- Las hormigas descubren y mantienen rutas.
- La feromona identifica rutas y determina las posibilidades de elegir una trayectoria.
- Las llamadas son “ubicadas” en rutas definidas por las hormigas.
- La tabla de feromonas es la tabla de ruteamiento. Se inicializa en forma random.
- Las hormigas son lanzadas regularmente a destinos random en la red.
- Cuando alcanzan su destino, las hormigas se sacan del circuito.

# OTRA (AJENA Y PROPIA)

- Empresas de logística como UPS y FedEx han explorado y aplicado algoritmos inspirados en colonias de hormigas para mejorar la eficiencia de sus operaciones de entrega, reduciendo costos y mejorando los tiempos de entrega. Este enfoque ayuda a abordar las complejidades logísticas asociadas con la planificación de rutas en entornos dinámicos y cambiantes.

# PROBLEMA DE HORARIOS

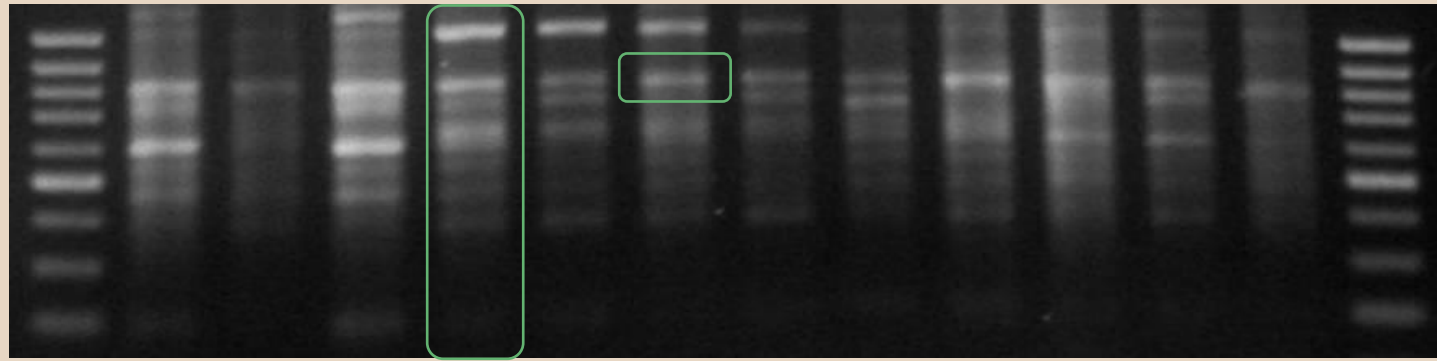


# PROCESAMIENTO DE IMÁGENES EN ESCALA DE GRISES

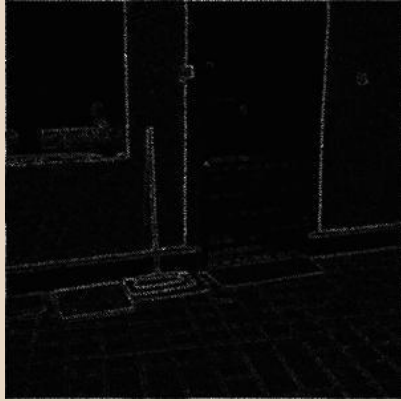
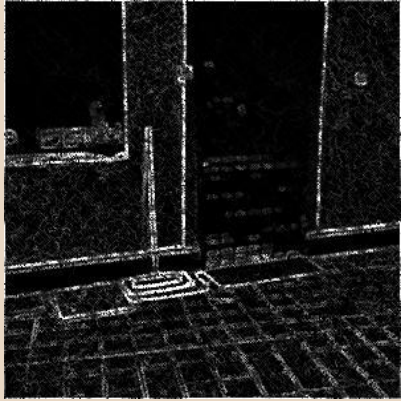
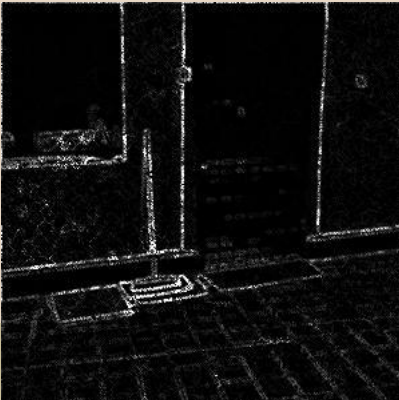
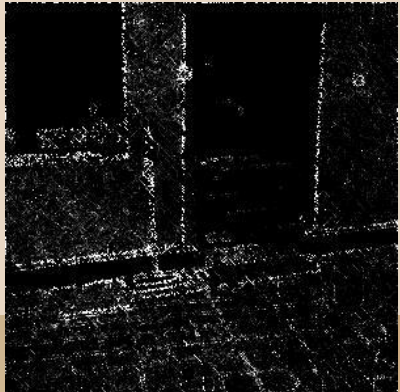
- Pía Maldonado

# EL PROBLEMA

- Imágenes RAPD (¡regresaron!)



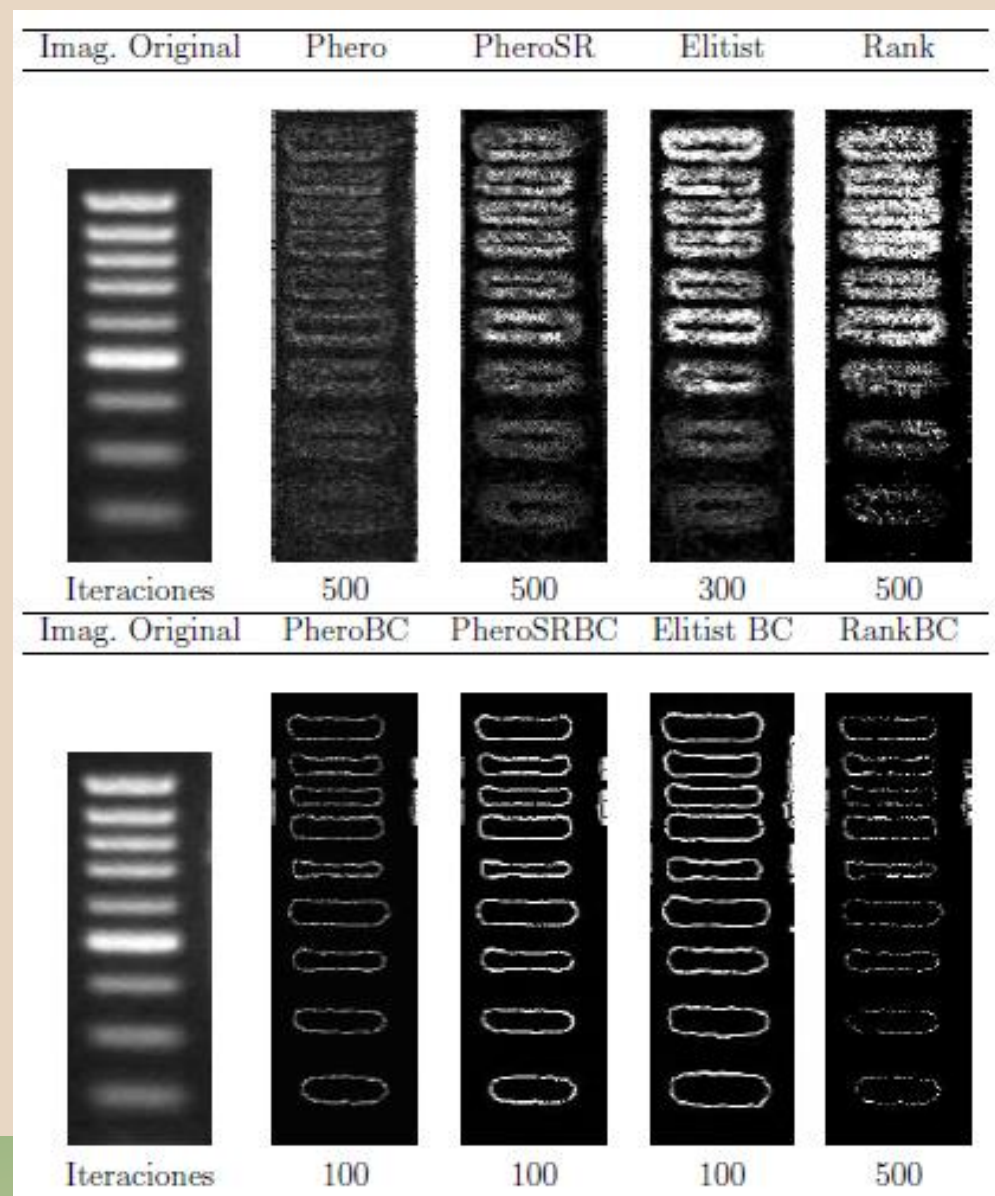
# RESULTADOS DE LOS ALGORITMOS

| Imagen Original                                                                     | Pherographia – 100 iteraciones                                                        | Pherographia sin Reproducción -250 iter.                                              |
|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|    |    |    |
|                                                                                     | Elitist – 100 iteraciones                                                             | Rank-Based-500 iteraciones                                                            |
|  |  |  |

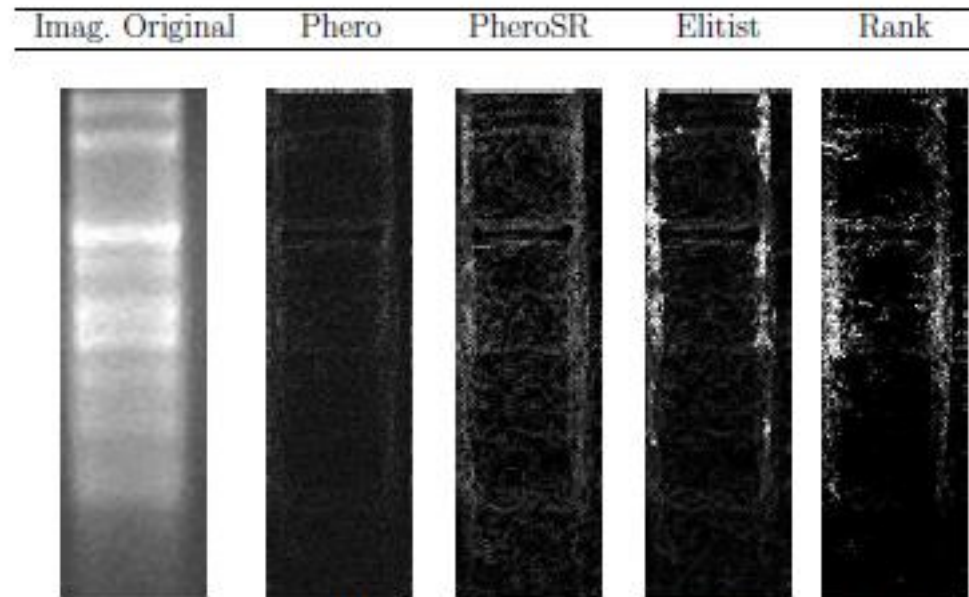
# PREPROCESAMIENTO

- Para las imágenes RAPD, se probó preprocesar las imágenes con el fin de destacar de mejor manera las bandas.
- Por eso se les aumentó el contraste a las imágenes y se binarizaron.

# RESULTADOS IMAGENES RAPD

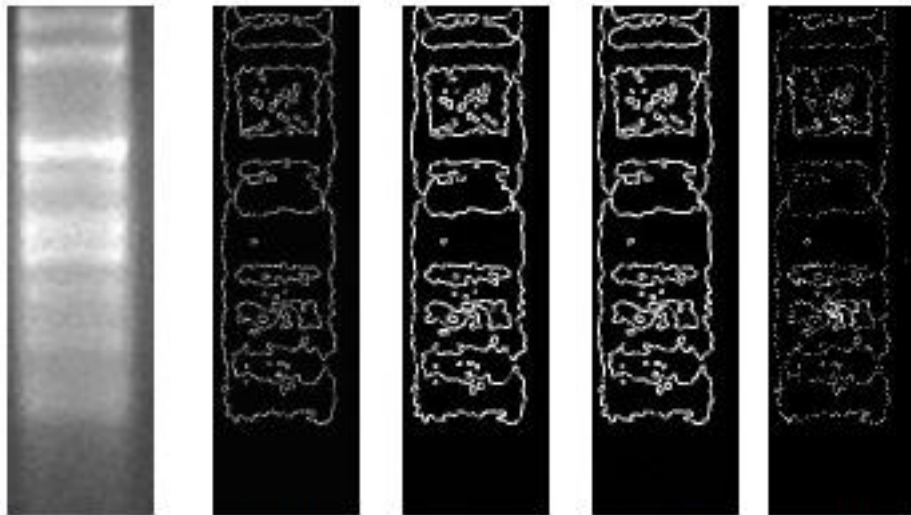


# RESULTADOS IMÁGENES RAPD



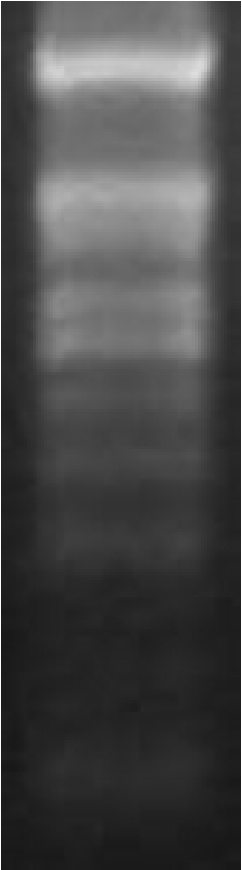
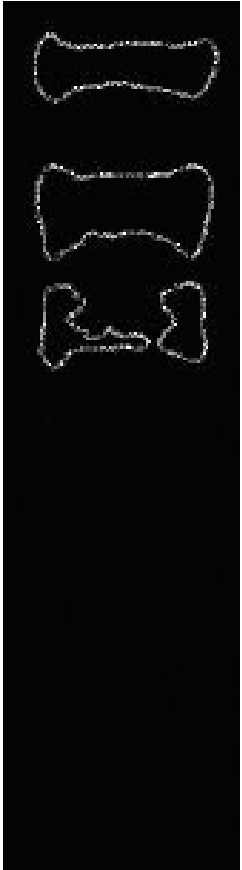
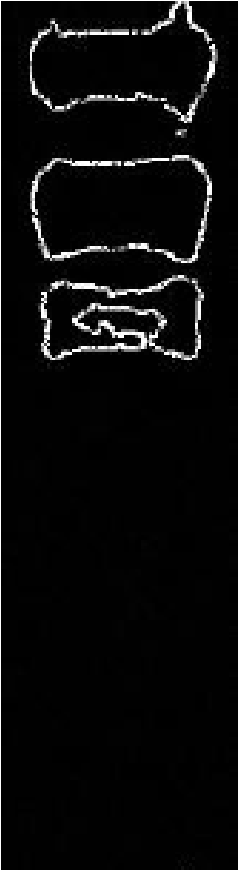
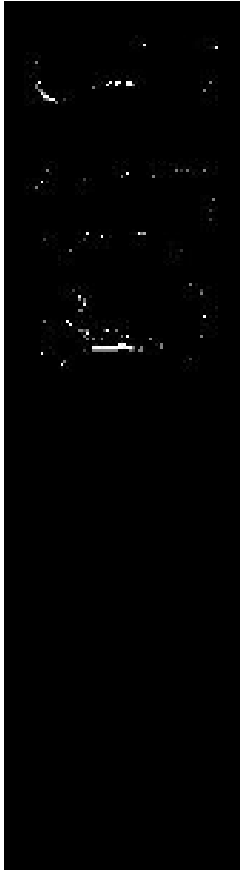
|             |     |     |     |     |
|-------------|-----|-----|-----|-----|
| Iteraciones | 500 | 500 | 300 | 500 |
|-------------|-----|-----|-----|-----|

| Imag. Original | PheroBC | PheroSRBC | Elitist BC | RankBC |
|----------------|---------|-----------|------------|--------|
|----------------|---------|-----------|------------|--------|



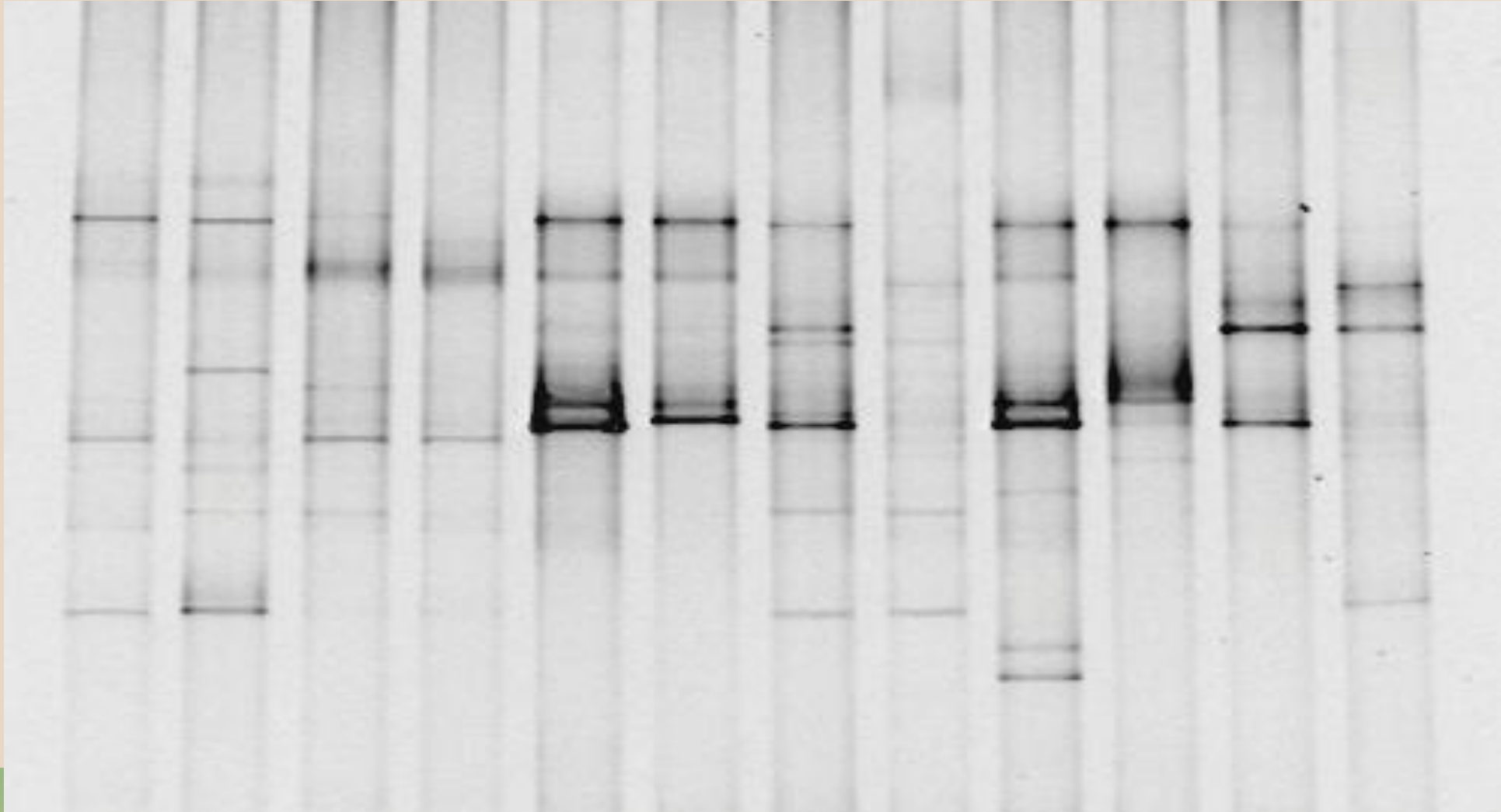
|             |     |     |     |     |
|-------------|-----|-----|-----|-----|
| Iteraciones | 100 | 100 | 100 | 500 |
|-------------|-----|-----|-----|-----|

# TIEMPOS DE EJECUCIÓN

| Imag. Original                                                                     | PheroBC                                                                             | PheroSRCB                                                                            | ElitistBC                                                                            | RankBC                                                                               |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
|  |  |  |  |  |
| Tiempo(min.)                                                                       | 104.8 min.                                                                          | 4.16 min.                                                                            | 4.3 min.                                                                             | 4.16 min.                                                                            |

# IDENTIFICACIÓN Y CUANTIFICACIÓN DE BANDAS EN IMÁGENES DGGE

- Carola Figueroa



LENA. LA  
ORIGINAL



# ALGUNAS PRUEBAS INICIALES



# DETECCIÓN DE BORDES USANDO ACO Y FUZZY LOGIC

- Sebastián Espinoza

original image



Final Image



# EN RESUMEN

- ACO es una meta-heurística competitiva para una gran variedad de problemas.
- A menudo requiere apoyo extra (por ejemplo, búsqueda local).
- Es necesario ir corrigiendo el balance entre intensificación y diversificación .
- Elementos aleatorios son importantes.
- Es necesario un análisis cuidadoso de los parámetros.
- Futuro – considerar estructuras alternativas.

# PROXIMA SEMANA

- Swarm Intelligence
  - Abejas artificiales